Team F

College0
Software Design Report
For College0

Version 2.0

Revision History

Date	Version	Description	Author
23/04/2026	2.0	2. Use Case Scenarios, 4. Detailed Design, 5. System Screens, 6. Group Meeting Memos	Karol Kopciuch
23/04/2026	2.0	2. Use Case Scenarios, 3. E-R Diagram for Entire System, 4. Detailed Design	Marcus Coppa
23/04/2026	2.0	1. Introduction, 2. Use Case Scenarios, 7. GitHub Repository	Momoka Yonashiro

Table of Contents

1. Introduction 4-5
   - 1.1 Purpose 4
   - 1.2 Collaboration Class Diagram 4-5
   - 1.3 Definitions, Acronyms, Clarifications 5
2. Use Case Scenarios 6-14
3. E-R Diagram for Entire System 15
4. Detailed Design 16-52
5. System Screens 53-56
   - 5.1 Major GUI Screens 53-55
   - 5.2 Sample Prototype 55-56
6. Group Meeting Memos 57
7. GitHub Repository 57

Software Design Report

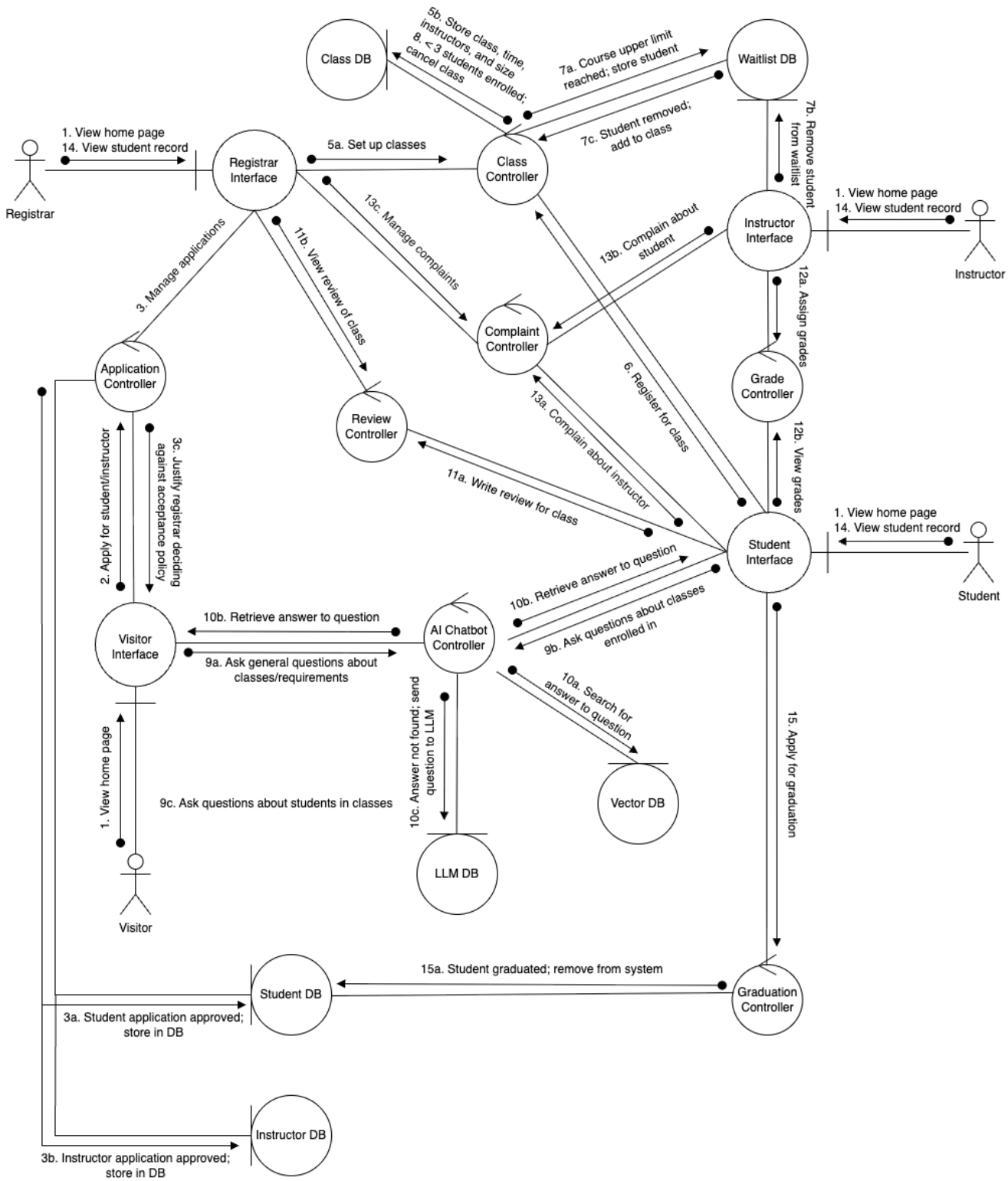
1. Introduction

1.1 Purpose

The purpose of this report is to provide the data structure and logic to carry out the functionalities of College0, an AI-enabled online college program management system. We will begin by explaining each of our use cases in Section 2, detailing both normal and exceptional scenarios paired with class diagrams and Petri-nets. In Section 3, we will display the distribution of data and the relationships between each class via an E-R diagram of the entire system. Afterwards, we will elaborate on our design in detail through pseudocode for every method in Section 4. Section 5 will demonstrate major GUI screens of the system and include a sample prototype of one of our functionalities, while Section 6 contains memos of our group meetings. Finally, Section 7 will provide the address of our GitHub repository, which will contain all of our materials including this report.

1.2 Collaboration Class Diagram

Below is a collaboration class diagram providing an overall picture of our system.



1.3 Definitions, Acronyms, Clarifications

Anywhere where the word “Professor” appears, it is synonymous with the word Instructor and vice versa. Please treat these words as one in the same.

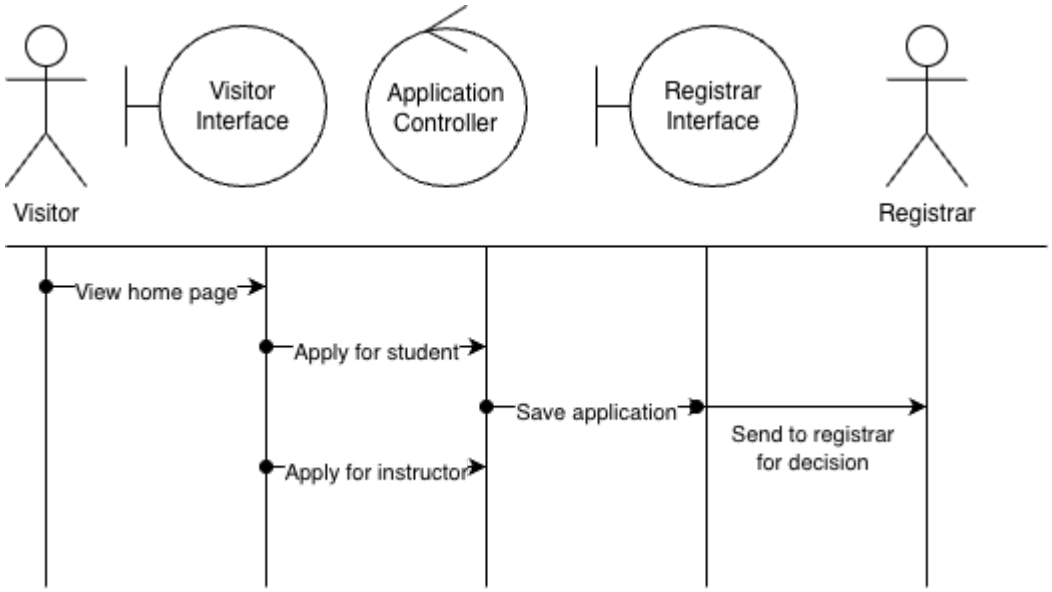
2. Use Case Scenarios

For each of the following use cases, we will describe normal and, when applicable, exceptional scenarios. As mentioned previously, each of these use cases will be accompanied by a diagram; either a sequential class diagram or a Petri net. Note that the use cases have been revised from our first report, the Software Requirements Specification, as we felt our current descriptions captured College0's functionalities more accurately.

Application for Student/Instructor Enrollment

Normal: Users designated as visitors apply to be either a student or instructor from their home page. The application is then sent to the registrar for a decision.

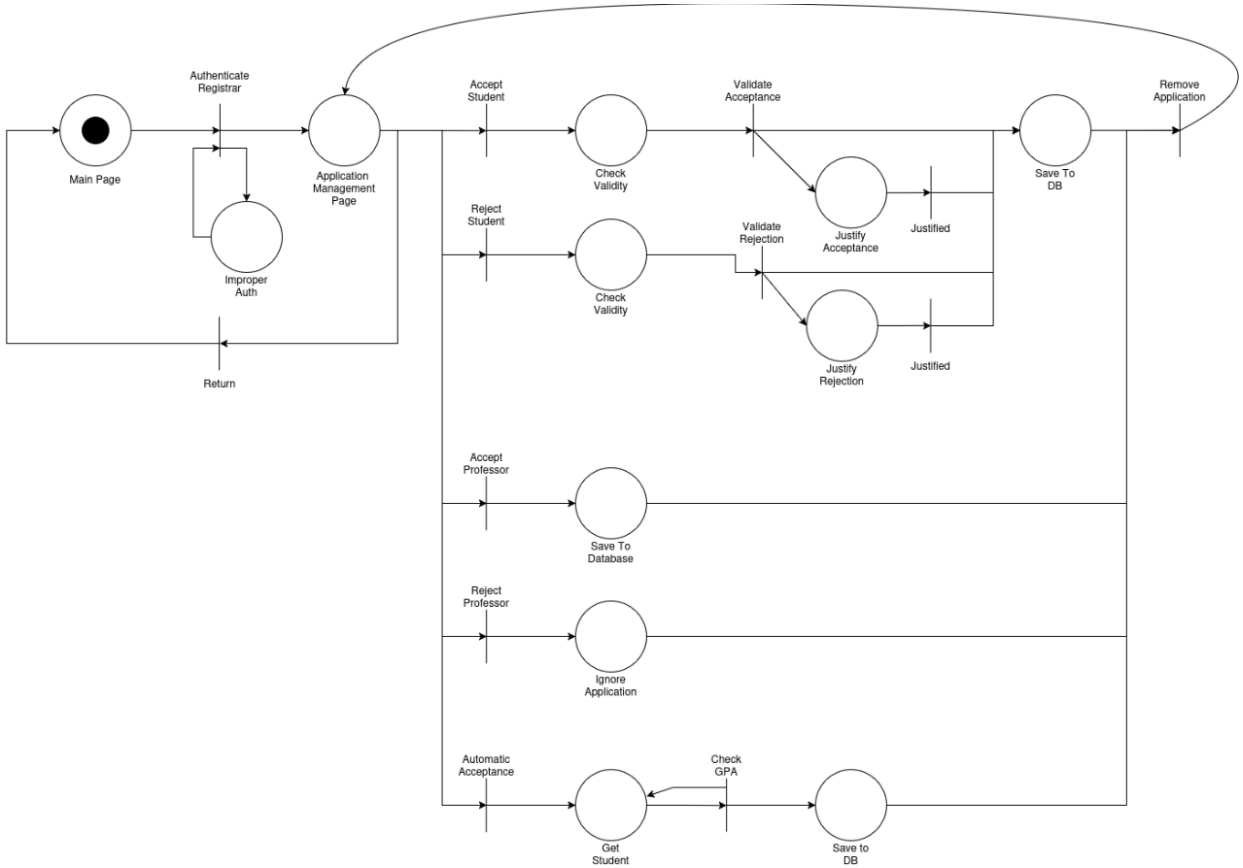
Exceptional: The applicant may provide insufficient or invalid data in their application.



Manage Applications

Normal: The registrar should accept any prospective student whose GPA is greater than 3.0 and whose program quota is not yet reached. Otherwise, they should reject the application. Accepted students will receive a student ID and password. Registrars can accept or reject instructors as they see fit, i.e., without the need for justification. Accepted instructors will be assigned classes.

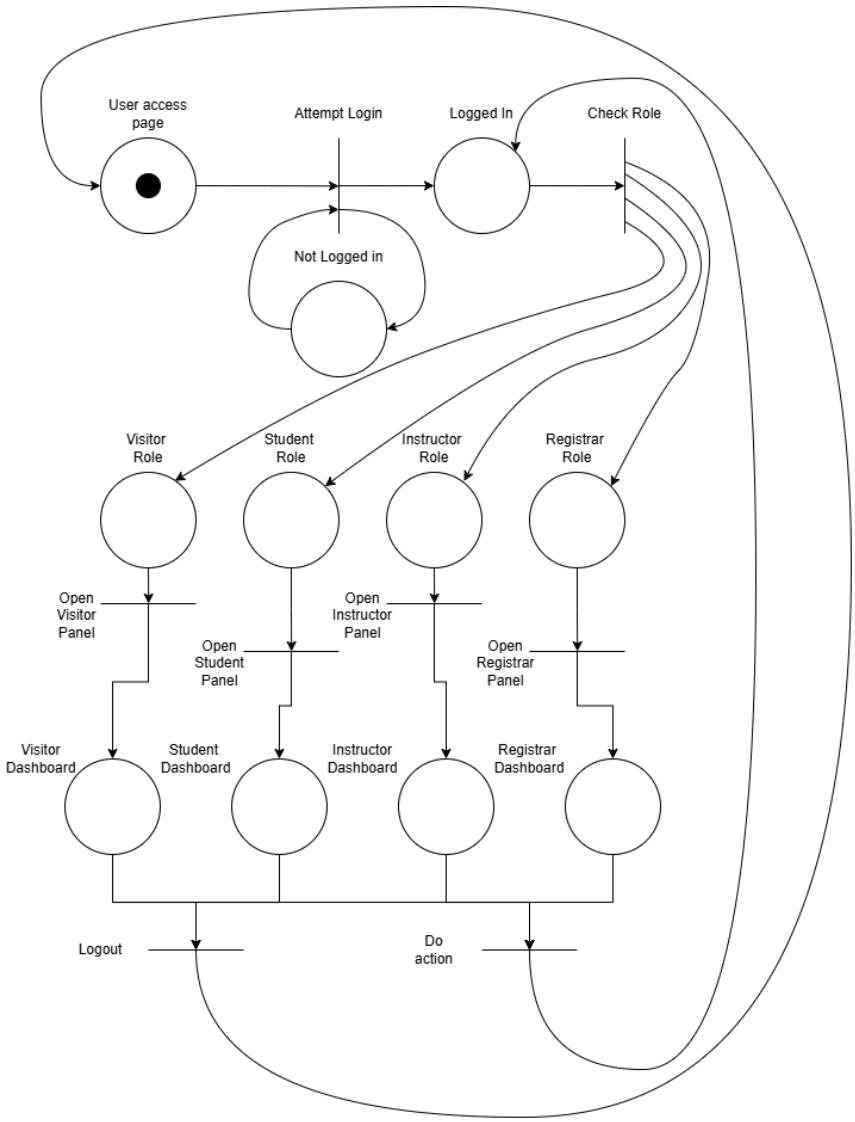
Exceptional: A registrar may decide against the GPA and program quote requirement rule. In this case, the registrar must justify themselves to the applicant.



View Home Page

Normal: User access home page with login credentials if none then they have only the permissions of a visitor.

Exceptional: Malicious actor attempts to brute force home page or AI scrapes the site.

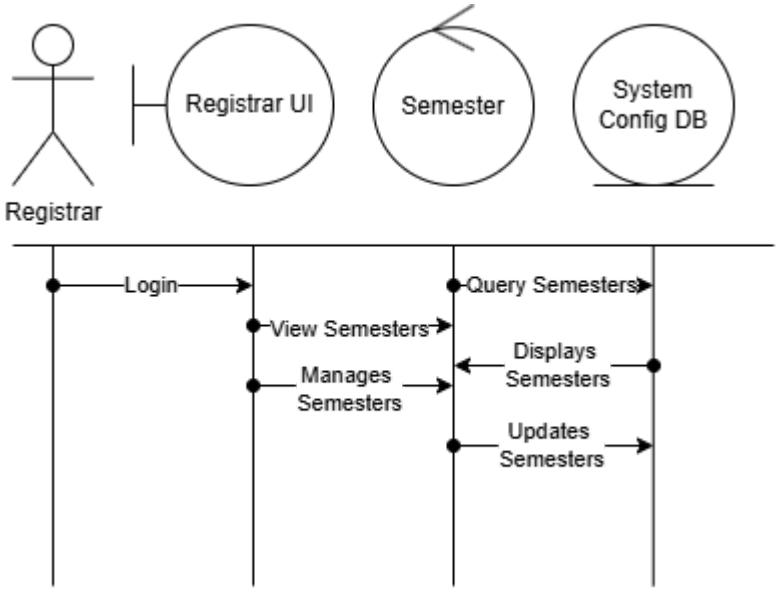


Academic Semester/ Course management

Manage Semesters

Normal: Registrar will have the ability to access the main semester management page allowing them to change the time of the semester.

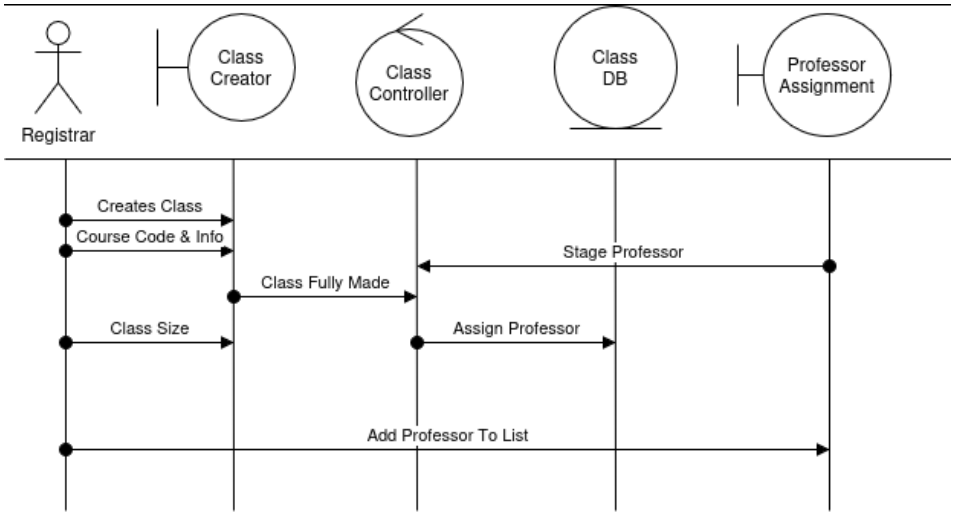
Exceptional: The semester could potentially be dropped into a null state creating a wildly disastrous bug at worst causing students to lose grades, and at best causing the professors unnecessary stress while grading student's work.



Setting Up Classes

Normal: Registrar logs in and creates classes assigning course code, id, session time, and instructor names.

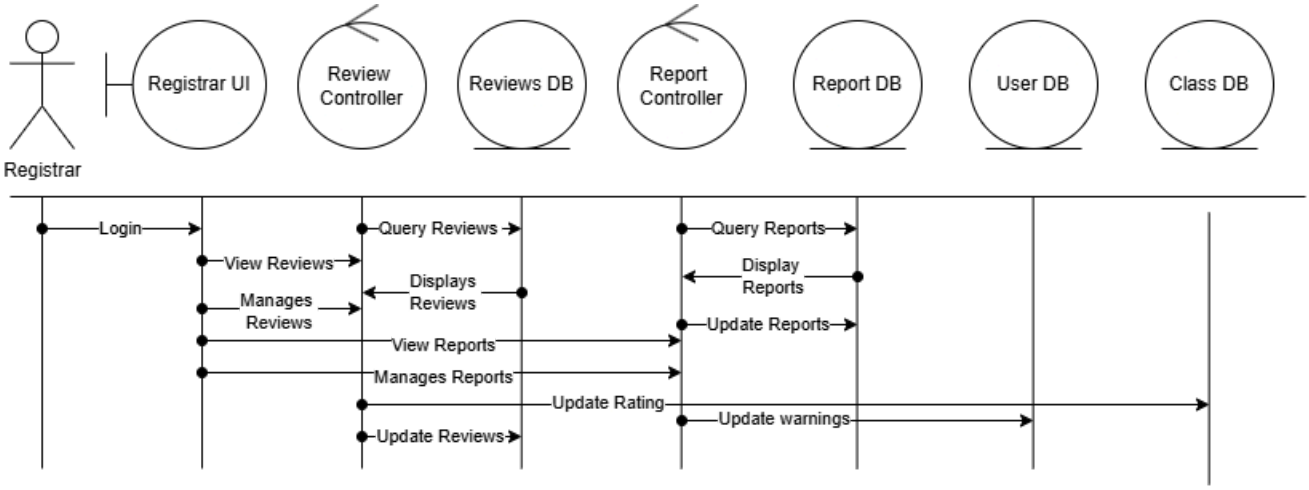
Exceptional: Registrar inputs incorrect times and makes classes at midnight.



View Reviews of Classes/ Reports of students or professors

Normal: Registrars will enter the review panel and can view the various different reviews/reports the students/professors have written giving them a good idea of what is going on.

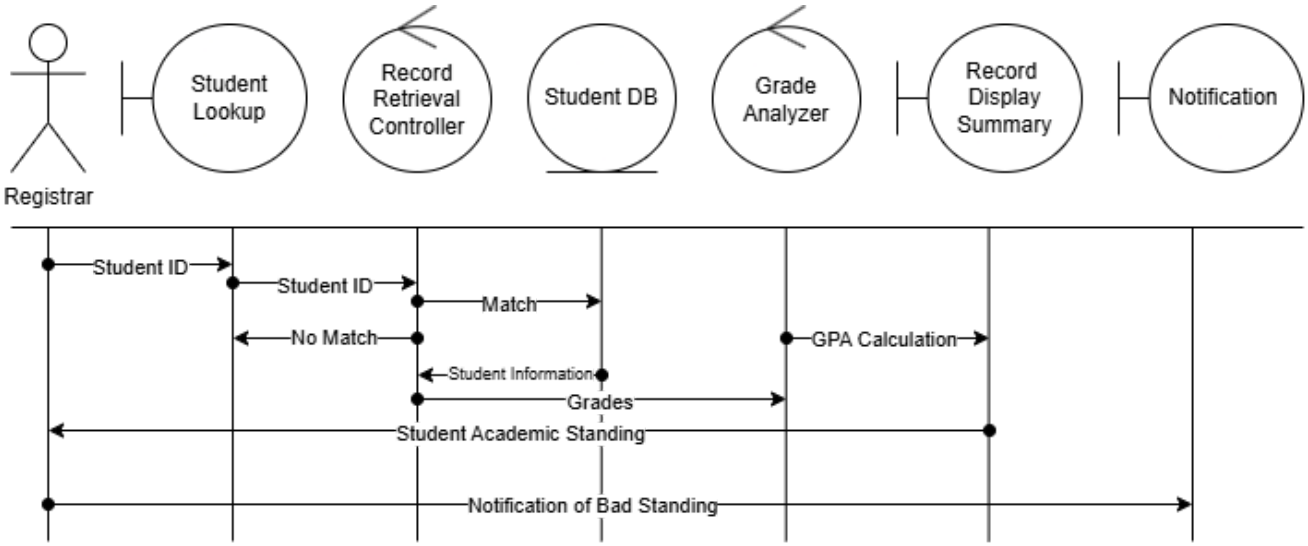
Exceptional: Potentially the Registrar may log in to find null data being returned as a result of some backend bug creating the issue of the proper number of reviews however not properly returning all of the necessary information such as the person who created the review/ when it was posted.



View Student Record

Normal: Registrar logs in and looks up students by ID , with automatic grade calculation displaying if the student is in good standing or not and giving a contact option.

Exceptional: Registrar looks up a non existent student ID in which they will be returned an error.

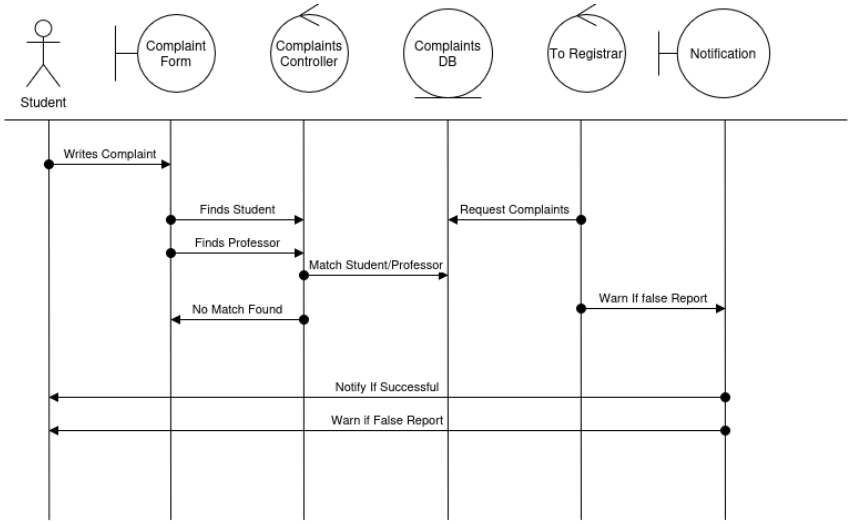


Complaints, Students and Professors

Student Complaints

Normal: Student/s write complaints about other students or instructors and will receive a notification if the report was successful or not. If it was found to be unfounded the writers can get a warning for making an unfounded report.

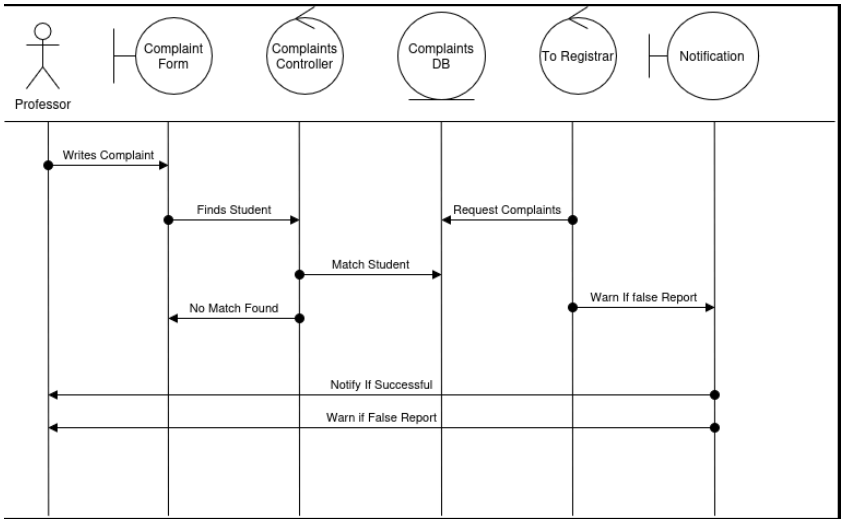
Exceptional: Student/s mass report other students or instructors leaving a mess for the registrar to go through.



Professor Complaints

Normal: Professors can write a complaint about any given student in the class that they are currently teaching and the complaint will be given to the registrar

Exceptional: Sometimes the webpage might bug out, and instead of properly filing the report it creates a null entry into the table causing invalid data to be returned and erroneous behaviour due to NULL data operations

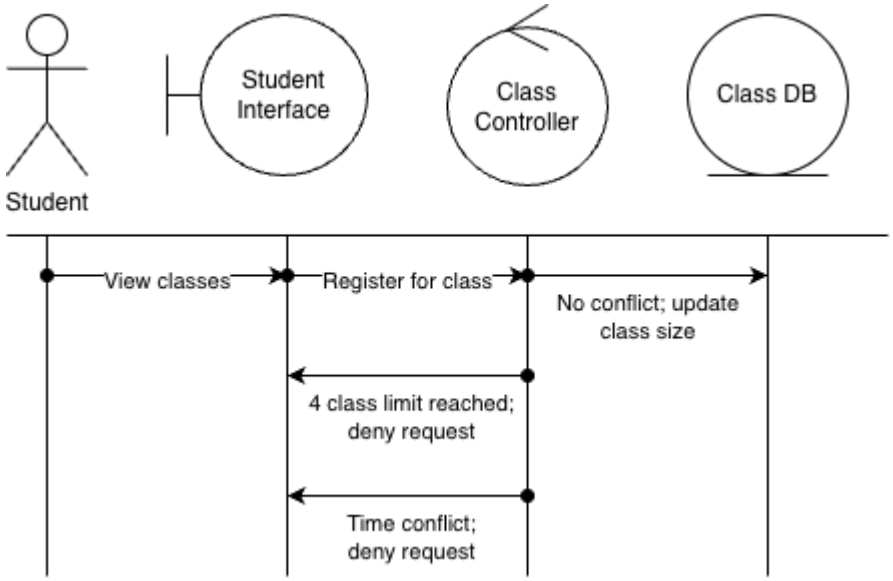


Classes

Class Registration

Normal: Students will be able to register for upwards of 4 different classes as long as none of the classes have a time conflict with each other

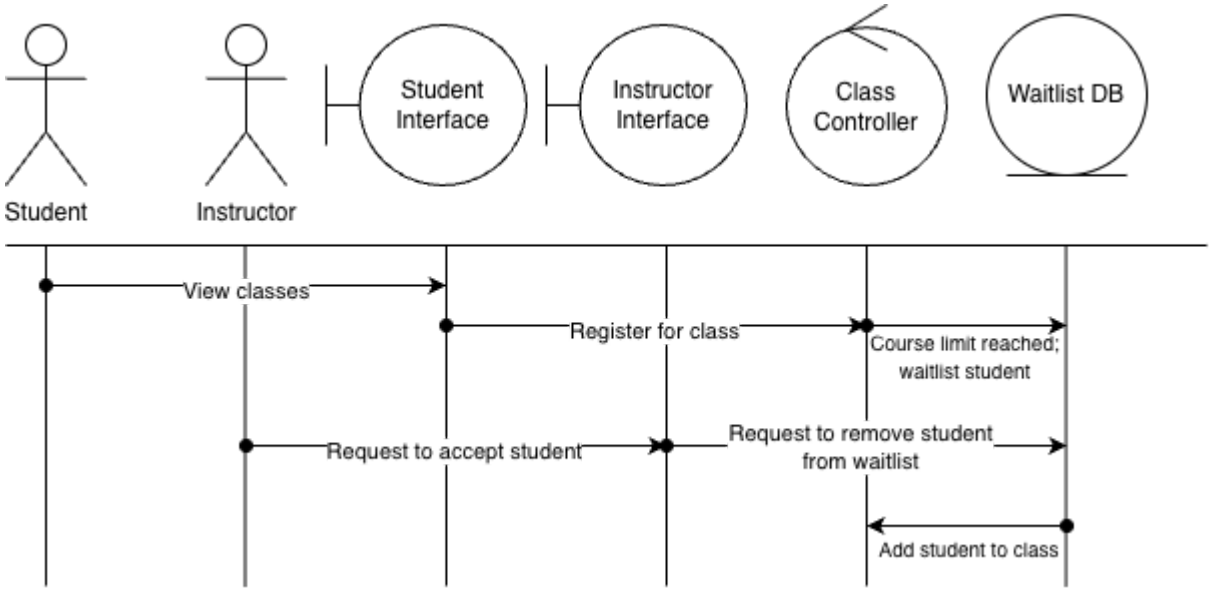
Exceptional: There may be a scenario in which a student signs up for a given class and as a result they will not be placed into the class properly on the backend, but still display that they are in the class, thus creating erroneous data logs on the college's end



Waitlist Registration / Management

Normal: Instructors can view the waitlist of their class and when a slot opens up they can select a student to come off the waitlist and join the class. The students can view said waitlist.

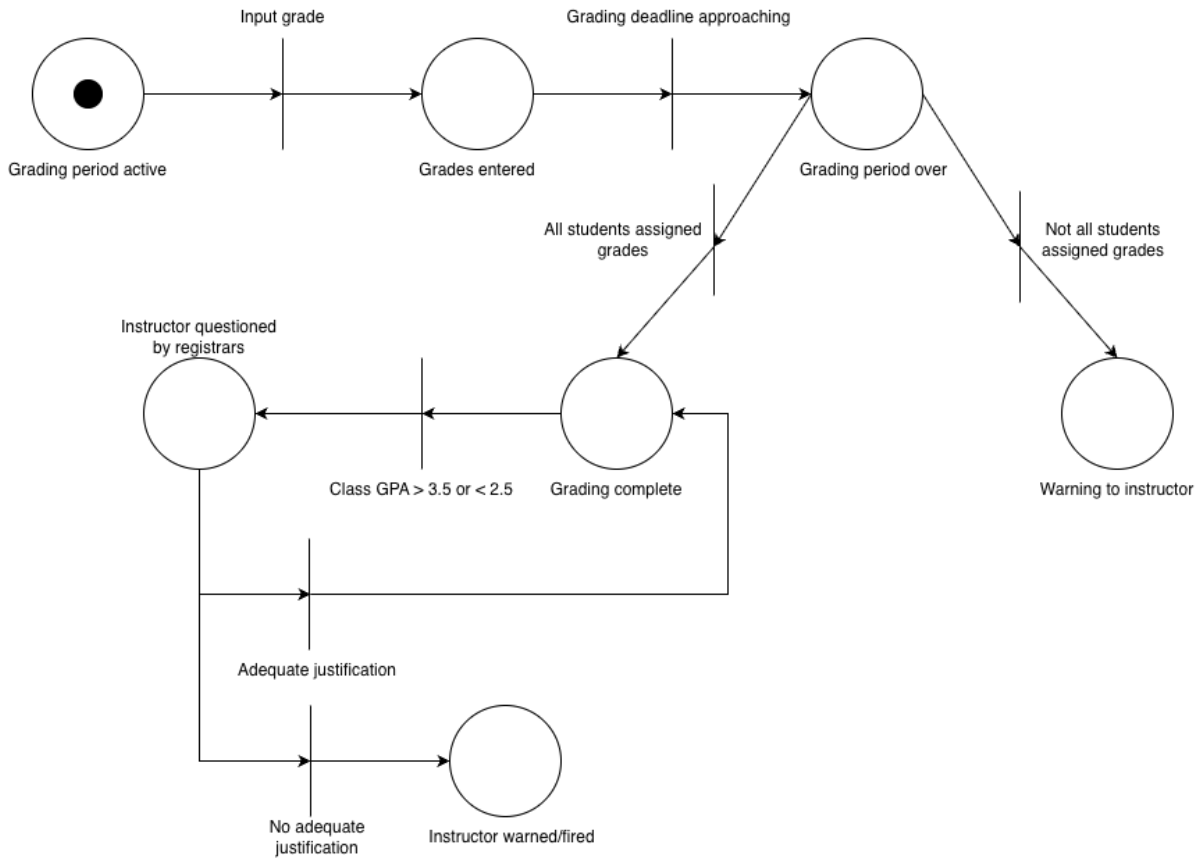
Exceptional: Mass students apply for a class overloading the Waitlist DB.



Grade Assignment

Normal: Instructor assigns grades during the grading period of the semester and instructors who fail to meet grading deadlines are given a warning.

Exceptional: Instructor fails to meet multiple deadlines resulting in the expulsion of the instructor.

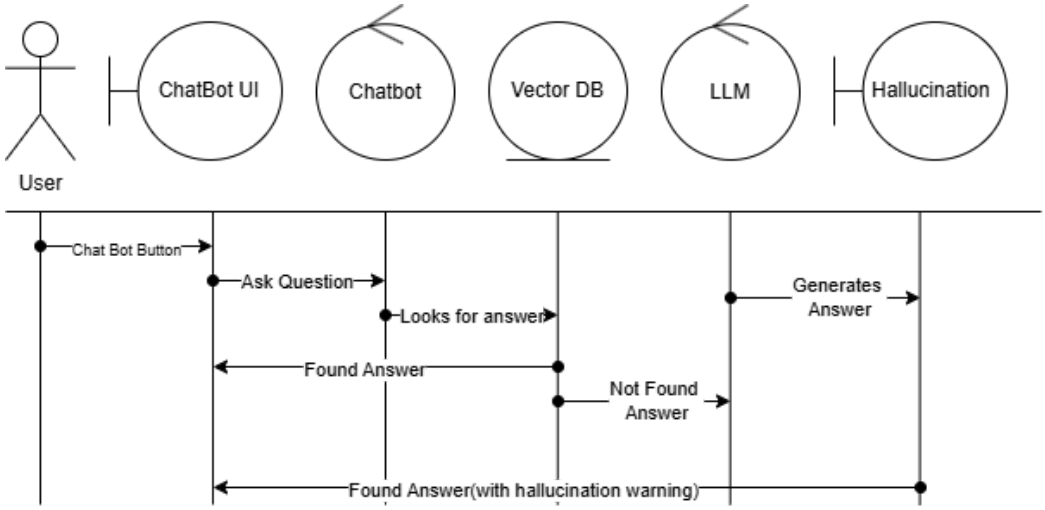


General System Functionality

ChatBot

Normal: Students, Visitors, Instructors and Registrars can ask our wonderful Artificial Intelligence various questions about the current semester and what their most recommended course of action should be for anything asked

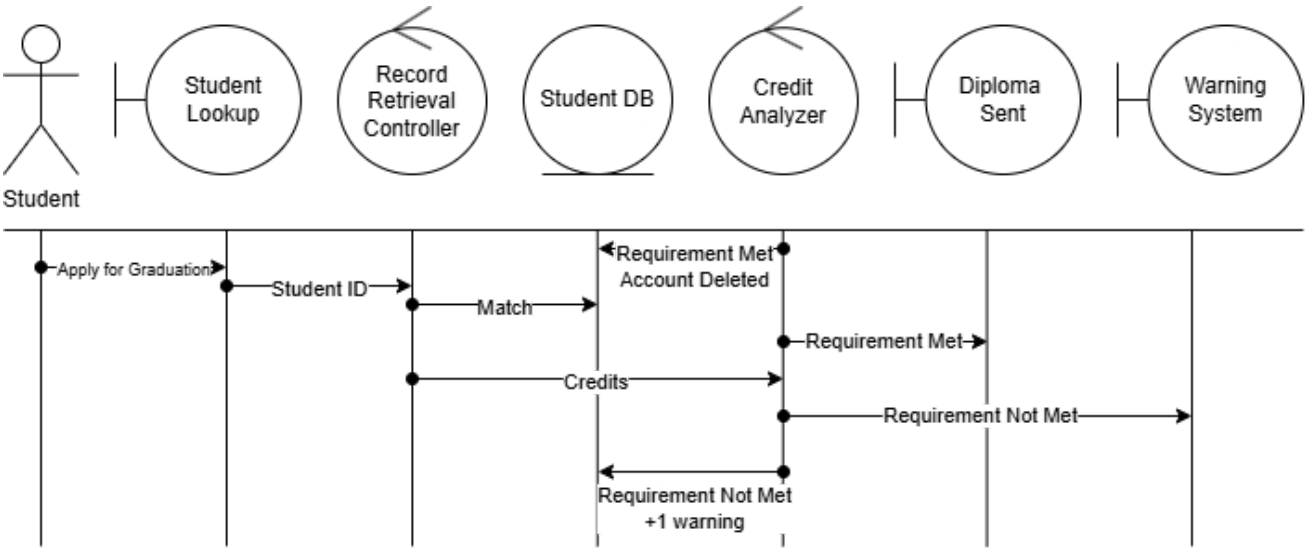
Exceptional: Potentially the AI might hallucinate and give users issues with various random tidbits of information that come from external sources and not from information available on the college page.



Graduation Application

Normal: The student has finished 8 classes and applies for graduation. The registrar finds the student has indeed completed all of their required courses. The student graduates and leaves the system with a Bachelor's degree.

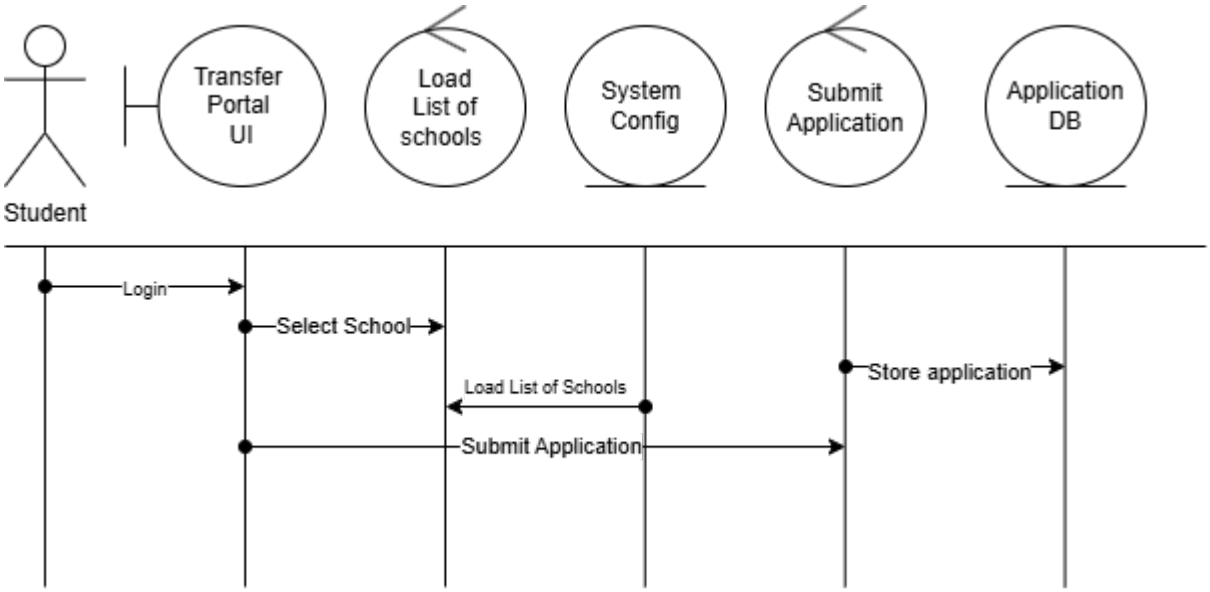
Exceptional: The registrar may find that the student has not completed all their required courses, in which case the student will receive a warning for reckless graduation application.



Transfer Application

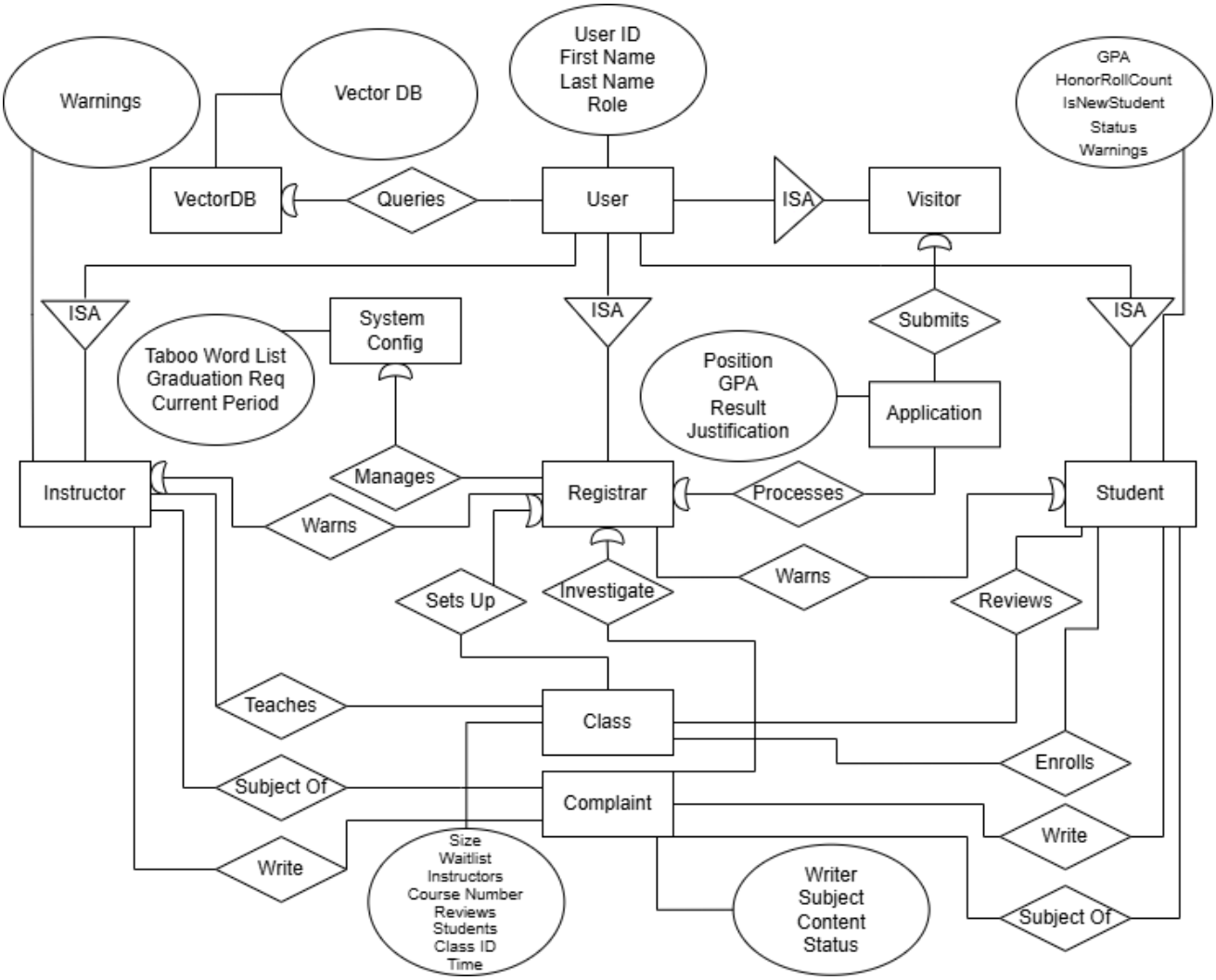
Normal: Student selects what school they would like to transfer to and submits an application.

Exceptional: Multiple students attempt to transfer out of a school and results in a school failing.



3. E-R Diagram for Entire System

Below is an E-R Diagram depicting the relational schema of our system:



4. Detailed Design

```
// INPUT:     username (String), password (String) from form fields
// OUTPUT:    Redirects to dashboard on success, displays error on failure
// FUNCTION:  Collects login fields and sends to backend, handles response status codes
FUNCTION handleLoginSubmit:
    username = getFieldValue("username")
    password = getFieldValue("password")

    IF username is empty OR password is empty:
        displayError("All fields required")
        RETURN

    response = SEND POST /api/login WITH { username, password }

    IF response.status == 200:
        redirectToDashboard(response.role)
    ELSE IF response.status == 401:
        displayError("Invalid credentials")
    ELSE IF response.status == 404:
        displayError("User not found")
    ELSE:
        displayError("Unexpected error")
END FUNCTION
```

```
// INPUT:     request { username (String), password (String) }
// OUTPUT:    HTTP status code, on success: { token (String), role (Enum) }
// FUNCTION:  Validates fields, looks up user, verifies password, returns session token and role
FUNCTION login(request): // Imagine this is spring boot or something like that
    username = request.username
    password = request.password

    IF username is null OR password is null:
        RETURN 400

    user = DB.findUserByUsername(username)

    IF user is null:
        RETURN 404

    IF NOT verifyPassword(password, user.hashedPassword):
        RETURN 401

    IF user.role NOT IN { VISITOR, STUDENT, INSTRUCTOR, REGISTRAR }:
        RETURN 403

    token = generateSessionToken(user.id, user.role)

    RETURN 200 WITH { token, user.role }
END FUNCTION
```

```
// INPUT:     plaintext (String), hashed (String)
// OUTPUT:    Boolean
// FUNCTION:  Compares a plaintext password against a stored bcrypt hash
FUNCTION verifyPassword(plaintext, hashed):
    RETURN compare(plaintext, hashed)
END FUNCTION
```

```
// INPUT:     username, password, firstName, lastName, address, targetMajor, GPA from form fields
// OUTPUT:    Redirects to dashboard on success, displays error on failure
// FUNCTION:  Collects student registration fields and sends to backend, handles response status codes
FUNCTION handleStudentSignUp:
    username    = getFieldValue("username")
    password    = getFieldValue("password")
    firstName   = getFieldValue("firstName")
    lastName    = getFieldValue("lastName")
    address     = getFieldValue("address")
    targetMajor = getFieldValue("targetMajor")
    GPA         = getFieldValue("GPA")

    IF ANY field is empty:
        displayError("All fields required")
        RETURN

    IF GPA is not a valid decimal between 0.0 and 4.0:
        displayError("Invalid GPA")
        RETURN

    response = SEND POST /api/signup/student WITH { username, password, firstName, lastName, address, targetMajor, GPA }

    IF response.status == 201:
        redirectToDashboard(STUDENT)
    ELSE IF response.status == 409:
        displayError("Username already taken")
    ELSE:
        displayError("Unexpected error")
END FUNCTION
```

```
// INPUT:     username, password, firstName, lastName, address, expertise, degree from form fields
// OUTPUT:    Redirects to dashboard on success, displays error on failure
// FUNCTION:  Collects instructor registration fields and sends to backend, handles response status codes
FUNCTION handleInstructorSignUp:
    username  = getFieldValue("username")
    password  = getFieldValue("password")
    firstName = getFieldValue("firstName")
    lastName  = getFieldValue("lastName")
    address   = getFieldValue("address")
    expertise = getFieldValue("expertise")
    degree    = getFieldValue("degree")

    IF ANY field is empty:
        displayError("All fields required")
        RETURN

    response = SEND POST /api/signup/instructor WITH { username, password, firstName, lastName, address, expertise, degree }

    IF response.status == 201:
        redirectToDashboard(INSTRUCTOR)
    ELSE IF response.status == 409:
        displayError("Username already taken")
    ELSE:
        displayError("Unexpected error")
END FUNCTION
```

```
// INPUT:     request { username, password, firstName, lastName, address, targetMajor, GPA }
// OUTPUT:    HTTP status code, on success: { token (String), role: STUDENT }
// FUNCTION:  Validates fields, checks for duplicate username, hashes password, stores new student record
FUNCTION signUpStudent(request):
    username    = request.username
    password    = request.password
    firstName   = request.firstName
    lastName    = request.lastName
    address     = request.address
    targetMajor = request.targetMajor
    GPA         = request.GPA

    IF ANY field is null:
        RETURN 400

    IF DB.findUserByUsername(username) is NOT null:
        RETURN 409

    hashedPassword = bcrypt.hash(password)

    newStudent = {
        username,
        hashedPassword,
        firstName,
        lastName,
        address,
        targetMajor,
        GPA,
        role: STUDENT
    }

    DB.saveUser(newStudent)

    token = generateSessionToken(newStudent.id, STUDENT)

    RETURN 201 WITH { token, role: STUDENT }
END FUNCTION
```

```
// INPUT:     request { username, password, firstName, lastName, address, targetMajor, GPA }
// OUTPUT:    HTTP status code, on success: { token (String), role: STUDENT }
// FUNCTION:  Validates fields, checks for duplicate username, hashes password, stores new student record
FUNCTION signUpStudent(request):
    username    = request.username
    password    = request.password
    firstName   = request.firstName
    lastName    = request.lastName
    address     = request.address
    targetMajor = request.targetMajor
    GPA         = request.GPA

    IF ANY field is null:
        RETURN 400

    IF DB.findUserByUsername(username) is NOT null:
        RETURN 409

    hashedPassword = bcrypt.hash(password)

    newStudent = {
        username,
        hashedPassword,
        firstName,
        lastName,
        address,
        targetMajor,
        GPA,
        role: STUDENT
    }

    DB.saveUser(newStudent)

    token = generateSessionToken(newStudent.id, STUDENT)

    RETURN 201 WITH { token, role: STUDENT }
END FUNCTION
```

```
// INPUT:        application - contains application.userId, application.GPA, application.sessionToken
//               programQuota - contains programQuota.limit and programQuota.currentCount
// OUTPUT:       success or error message
// FUNCTIONALITY: Checks the GPA field and program quota of an incoming application,
//               if GPA is above 3.0 AND the program quota has not been reached automatically
//               accepts the application, otherwise automatically rejects it
FUNCTION autoAcceptApplication(application, programQuota):

    IF application is NULL:
        RETURN error "Invalid application"

    authLevel = returnAuthLevel(application.sessionToken)

    IF authLevel != REGISTRAR:
        RETURN error "Unauthorized - Registrar access required"

    IF application.GPA is NULL:
        RETURN error "GPA field is missing"

    IF programQuota is NULL:
        RETURN error "Program quota data is missing"

    quotaReached = programQuota.currentCount >= programQuota.limit

    IF application.GPA > 3.0 AND quotaReached == FALSE:
        SEND moveStudentToMainDatabase(application.userId)
        SEND notifyStudent(application.userId, "ACCEPTED")
        RETURN success "Application auto accepted - Student added to main database"

    ELSE IF application.GPA <= 3.0:
        SEND deletePendingApplication(application.userId)
        SEND notifyStudent(application.userId, "REJECTED")
        RETURN success "Application auto rejected - GPA below required 3.0"

    ELSE IF quotaReached == TRUE:
        SEND deletePendingApplication(application.userId)
        SEND notifyStudent(application.userId, "REJECTED")
        RETURN success "Application auto rejected - Program quota has been reached"

END FUNCTION
```

```
// INPUT:        authLevel - the role of the current user
//               applicationList - list of pending applications
// OUTPUT:       rendered list of applications with accept/reject buttons
// FUNCTIONALITY: Displays the list of pending applications, each with an
//               accept and reject button, only visible to REGISTRAR
FUNCTION displayApplications(authLevel, applicationList):

    IF authLevel != REGISTRAR:
        DISPLAY error "Unauthorized - Registrar access required"
        RETURN

    IF applicationList is NULL or empty:
        DISPLAY "No pending applications"
        RETURN

    FOR EACH application IN applicationList:
        DISPLAY application.name
        DISPLAY application.GPA
        DISPLAY application.status

        DISPLAY button "Accept" -> onClick SEND processApplicationDecision(application, "ACCEPTED")
        DISPLAY button "Reject" -> onClick SEND processApplicationDecision(application, "REJECTED")

    END FOR

END FUNCTION
```

```
// INPUT:        userId - the ID of the applicant to be moved
// OUTPUT:       success or error message
// FUNCTIONALITY: Pulls the applicants info from the pending applications table,
//               saves it into the primary student database, then deletes the pending entry
FUNCTION moveStudentToMainDatabase(userId):

    IF userId is NULL:
        RETURN error "Invalid userId"

    studentData = fetchPendingApplication(userId)

    IF studentData is NULL or empty:
        RETURN error "No pending application found for user"

    result = insertIntoMainDatabase(studentData)

    IF result is NULL or failed:
        RETURN error "Failed to insert student into main database"

    SEND deletePendingApplication(userId)

    RETURN success "Student successfully moved to main database"

END FUNCTION
```

```
// INPUT:        sessionToken - the token of the current session
// OUTPUT:       AuthLevel - VISITOR, STUDENT, INSTRUCTOR, REGISTRAR
// FUNCTIONALITY: Checks the session token and returns the role of the current user
FUNCTION returnAuthLevel(sessionToken):
    IF sessionToken is NULL or expired:
        RETURN VISITOR

    user = findUserByToken(sessionToken)

    IF user is NULL:
        RETURN VISITOR
    ELSE IF user has role "STUDENT":
        RETURN STUDENT
    ELSE IF user has role "INSTRUCTOR":
        RETURN INSTRUCTOR
    ELSE IF user has role "REGISTRAR":
        RETURN REGISTRAR
    ELSE:
        RETURN VISITOR
END FUNCTION
```

```
// INPUT:        request - contains request.type, request.sessionToken, request.userId, request.filters
// OUTPUT:       data - the requested data, or an error message
// FUNCTIONALITY: Validates the request, checks authorization, fetches and returns the appropriate data
FUNCTION returnRequestedData(request):
    IF request or request.type is NULL:
        RETURN error "Invalid request"

    authLevel = returnAuthLevel(request.sessionToken)

    IF user is not authorized for request.type:
        RETURN error "Unauthorized"

    IF request.type == "USER_PROFILE":
        data = fetchUserProfile(request.userId)
    ELSE IF request.type == "FEED":
        data = fetchDefaultFeed(authLevel)
    ELSE IF request.type == "STATS":
        data = fetchStats(request.filters)
    ELSE:
        RETURN error "Unknown request type"

    IF data is NULL:
        RETURN error "No data found"

    RETURN data
END FUNCTION
```

```
// INPUT:        authLevel - the role of the current user (VISITOR, STUDENT, INSTRUCTOR, REGISTRAR)
// OUTPUT:       rendered navbar and landing page view
// FUNCTIONALITY: Builds and renders the correct navbar and landing page based on the user's role
FUNCTION displayNavbarAndLandingPage(authLevel):
    IF authLevel == VISITOR:
        navbar = ["Home", "Login", "Sign Up"]
        landingPage = guestView
    ELSE IF authLevel == STUDENT:
        navbar = ["Home", "My Courses", "Schedule", "Profile", "Logout"]
        landingPage = studentView
    ELSE IF authLevel == INSTRUCTOR:
        navbar = ["Home", "My Courses", "Gradebook", "Profile", "Logout"]
        landingPage = instructorView
    ELSE IF authLevel == REGISTRAR:
        navbar = ["Home", "Manage Courses", "Manage Users", "Reports", "Logout"]
        landingPage = registrarView

    RENDER navbar
    RENDER landingPage
END FUNCTION
```

```
// INPUT:        update - contains update.type, update.sessionToken, update.payload
// OUTPUT:       success or error message
// FUNCTIONALITY: Listens for incoming frontend update requests, validates and authenticates them,
//               then routes to the correct update function based on the users role and update type
FUNCTION sendUpdateToBackend(update):

    IF update is NULL:
        RETURN error "Invalid update"

    authLevel = returnAuthLevel(update.sessionToken)

    IF authLevel == VISITOR:
        RETURN error "Unauthorized - Please log in"

    IF update.type is NULL:
        RETURN error "No update type specified"

    IF update.payload is NULL or empty:
        RETURN error "No data provided to update"

    // STUDENT updates
    IF authLevel == STUDENT:
        IF update.type == "ENROLL_COURSE":
            SEND enrollInCourse(update.userId, update.payload.courseId)
        ELSE IF update.type == "DROP_COURSE":
            SEND dropCourse(update.userId, update.payload.courseId)
        ELSE IF update.type == "UPDATE_PROFILE":
            SEND updateProfile(update.userId, update.payload)
        ELSE:
            RETURN error "Unauthorized update type for STUDENT"

    // INSTRUCTOR updates
    ELSE IF authLevel == INSTRUCTOR:
        IF update.type == "UPDATE_GRADE":
            SEND updateGrade(update.userId, update.payload.studentId, update.payload.grade)
        ELSE IF update.type == "UPDATE_COURSE":
            SEND updateCourse(update.userId, update.payload.courseId, update.payload)
        ELSE IF update.type == "UPDATE_PROFILE":
            SEND updateProfile(update.userId, update.payload)
        ELSE:
            RETURN error "Unauthorized update type for INSTRUCTOR"

    // REGISTRAR updates
    ELSE IF authLevel == REGISTRAR:
        IF update.type == "ADD_COURSE":
            SEND addCourse(update.payload)
        ELSE IF update.type == "REMOVE_COURSE":
            SEND removeCourse(update.payload.courseId)
        ELSE IF update.type == "ADD_USER":
            SEND addUser(update.payload)
        ELSE IF update.type == "REMOVE_USER":
            SEND removeUser(update.payload.userId)
        ELSE IF update.type == "UPDATE_ENROLLMENT":
            SEND updateEnrollment(update.payload.studentId, update.payload.courseId)
        ELSE:
            RETURN error "Unauthorized update type for REGISTRAR"

    RETURN success "Update completed successfully"

END FUNCTION
```

```
// INPUT:        request - contains request.sessionToken, request.semesterId, request.newPeriod
// OUTPUT:       success or error message
// FUNCTIONALITY: Validates the registrar's authority, then updates the semester's
//               current time period to the new period provided
FUNCTION changeSemesterPeriod(request):

    IF request is NULL:
        RETURN error "Invalid request"

    authLevel = returnAuthLevel(request.sessionToken)

    IF authLevel != REGISTRAR:
        RETURN error "Unauthorized - Registrar access required"

    IF request.semesterId is NULL:
        RETURN error "No semester specified"

    IF request.newPeriod is NULL:
        RETURN error "No period specified"

    IF request.newPeriod NOT IN ["SET_UP", "REGISTRATION", "CURRENTLY_RUNNING", "GRADING"]:
        RETURN error "Invalid period - must be SET_UP, REGISTRATION, CURRENTLY_RUNNING, or GRADING"

    semester = fetchSemester(request.semesterId)

    IF semester is NULL:
        RETURN error "Semester not found"

    result = updateSemesterPeriod(request.semesterId, request.newPeriod)

    IF result is NULL or failed:
        RETURN error "Failed to update semester period"

    RETURN success "Semester period updated to " + request.newPeriod

END FUNCTION
```

```
// INPUT:        request - contains request.sessionToken, request.courseCode,
//               request.courseName, request.room, request.days
// OUTPUT:       success or error message
// FUNCTIONALITY: Validates the registrar's authority, checks for duplicate course codes,
//               then creates and saves the new class entry to the database
FUNCTION establishClass(request):

    IF request is NULL:
        RETURN error "Invalid request"

    authLevel = returnAuthLevel(request.sessionToken)

    IF authLevel != REGISTRAR:
        RETURN error "Unauthorized - Registrar access required"

    IF request.courseCode is NULL:
        RETURN error "Course code is required"

    IF request.courseName is NULL:
        RETURN error "Course name is required"

    IF request.room is NULL:
        RETURN error "Room is required"

    IF request.days is NULL or empty:
        RETURN error "Days are required"

    existingClass = fetchClassByCourseCode(request.courseCode)

    IF existingClass is NOT NULL:
        RETURN error "Course code already exists"

    newClass = {
        courseCode : request.courseCode,
        courseName : request.courseName,
        room       : request.room,
        days       : request.days,
        instructor : NULL,
        status     : "UNASSIGNED"
    }

    result = insertClassIntoDatabase(newClass)

    IF result is NULL or failed:
        RETURN error "Failed to create class"

    RETURN success "Class " + request.courseCode + " successfully created"

END FUNCTION
```

```
// INPUT:        request - contains request.sessionToken, request.courseCode, request.instructor
// OUTPUT:       success or error message
// FUNCTIONALITY: Validates the registrar's authority, checks that the instructor exists and
//               that their expertise matches the course, then assigns them to the class
FUNCTION assignInstructorToClass(request):

    IF request is NULL:
        RETURN error "Invalid request"

    authLevel = returnAuthLevel(request.sessionToken)

    IF authLevel != REGISTRAR:
        RETURN error "Unauthorized - Registrar access required"

    IF request.courseCode is NULL:
        RETURN error "No course specified"

    IF request.instructorId is NULL:
        RETURN error "No instructor specified"

    class = fetchClassByCourseCode(request.courseCode)

    IF class is NULL:
        RETURN error "Class not found"

    instructor = fetchInstructorById(request.instructorId)

    IF instructor is NULL:
        RETURN error "Instructor not found"

    IF instructor.expertise does NOT match class.courseCode or class.courseName:
        RETURN error "Instructor expertise does not match the course"

    IF class.instructor is NOT NULL:
        RETURN error "Class already has an assigned instructor"

    result = updateClassInstructor(request.courseCode, request.instructorId)

    IF result is NULL or failed:
        RETURN error "Failed to assign instructor"

    RETURN success "Instructor " + instructor.name + " assigned to " + request.courseCode

END FUNCTION
```

```
// INPUT:        authLevel - the role of the current user
//               semester - contains semester.id and semester.currentPeriod
// OUTPUT:       rendered semester period management panel
// FUNCTIONALITY: Displays the current semester period and buttons to switch
//               to any of the four periods, only visible to REGISTRAR
FUNCTION displaySemesterManagement(authLevel, semester):

    IF authLevel != REGISTRAR:
        DISPLAY error "Unauthorized - Registrar access required"
        RETURN

    IF semester is NULL:
        DISPLAY error "No semester data found"
        RETURN

    DISPLAY "Current Period: " + semester.currentPeriod

    DISPLAY button "Set-Up"            -> onClick SEND changeSemesterPeriod(semester.id, "SET_UP")
    DISPLAY button "Registration"      -> onClick SEND changeSemesterPeriod(semester.id, "REGISTRATION")
    DISPLAY button "Currently Running" -> onClick SEND changeSemesterPeriod(semester.id, "CURRENTLY_RUNNING")
    DISPLAY button "Grading"           -> onClick SEND changeSemesterPeriod(semester.id, "GRADING")

END FUNCTION
```

```
// INPUT:        authLevel - the role of the current user
// OUTPUT:       rendered form to create a new class
// FUNCTIONALITY: Displays input fields for course code, name, room, and days,
//               then sends the new class data to the backend on submit,
//               only visible to REGISTRAR
FUNCTION displayEstablishClass(authLevel):

    IF authLevel != REGISTRAR:
        DISPLAY error "Unauthorized - Registrar access required"
        RETURN

    DISPLAY input field "Course Code"
    DISPLAY input field "Course Name"
    DISPLAY input field "Room"
    DISPLAY input field "Days"

    DISPLAY button "Create Class" -> onClick SEND establishClass(courseCode, courseName, room, days)

END FUNCTION
```

```
// INPUT:        authLevel - the role of the current user
//               classList - list of existing classes
//               instructorList - list of instructors filtered by expertise
// OUTPUT:       rendered panel to assign an instructor to a class
// FUNCTIONALITY: Displays a dropdown of existing classes, and a filtered dropdown
//               of instructors whose expertise matches the selected class,
//               then sends the assignment to the backend on submit,
//               only visible to REGISTRAR
FUNCTION displayAssignInstructor(authLevel, classList, instructorList):

    IF authLevel != REGISTRAR:
        DISPLAY error "Unauthorized - Registrar access required"
        RETURN

    IF classList is NULL or empty:
        DISPLAY error "No classes available"
        RETURN

    IF instructorList is NULL or empty:
        DISPLAY error "No instructors available"
        RETURN

    DISPLAY dropdown "Select Class" -> options FROM classList

    ON classSelected:
        filteredInstructors = FILTER instructorList WHERE instructor.expertise matches selectedClass
        DISPLAY dropdown "Select Instructor" -> options FROM filteredInstructors

    DISPLAY button "Assign Instructor" -> onClick SEND assignInstructorToClass(selectedClass, selectedInstructor)

END FUNCTION
```

```
// INPUT:        request - contains request.sessionToken, request.targetId,
//               request.targetType, request.reportDetails
// OUTPUT:       success or error message
// FUNCTIONALITY: Allows a student to submit a report against a professor or another student,
//               validates the session and target, then saves the report to the database
FUNCTION submitReportAsStudent(request):

    IF request is NULL:
        RETURN error "Invalid request"

    authLevel = returnAuthLevel(request.sessionToken)

    IF authLevel != STUDENT:
        RETURN error "Unauthorized - Student access required"

    IF request.targetId is NULL:
        RETURN error "No target specified"

    IF request.targetType NOT IN ["PROFESSOR", "STUDENT"]:
        RETURN error "Invalid target type - must be PROFESSOR or STUDENT"

    IF request.reportDetails is NULL or empty:
        RETURN error "Report details are required"

    target = fetchUserById(request.targetId)

    IF target is NULL:
        RETURN error "Target user not found"

    newReport = {
        reporterId   : request.userId,
        reporterType : "STUDENT",
        targetId     : request.targetId,
        targetType   : request.targetType,
        details      : request.reportDetails,
        status       : "PENDING"
    }

    result = insertReportIntoDatabase(newReport)

    IF result is NULL or failed:
        RETURN error "Failed to submit report"

    RETURN success "Report successfully submitted"

END FUNCTION
```

```
// INPUT:        request - contains request.sessionToken, request.targetId,
//               request.reportDetails
// OUTPUT:       success or error message
// FUNCTIONALITY: Allows an instructor to submit a report against a student,
//               validates the session and target, then saves the report to the database
FUNCTION submitReportAsInstructor(request):

    IF request is NULL:
        RETURN error "Invalid request"

    authLevel = returnAuthLevel(request.sessionToken)

    IF authLevel != INSTRUCTOR:
        RETURN error "Unauthorized - Instructor access required"

    IF request.targetId is NULL:
        RETURN error "No target specified"

    IF request.reportDetails is NULL or empty:
        RETURN error "Report details are required"

    target = fetchUserById(request.targetId)

    IF target is NULL:
        RETURN error "Target student not found"

    IF target.role != "STUDENT":
        RETURN error "Instructors can only report students"

    newReport = {
        reporterId   : request.userId,
        reporterType : "INSTRUCTOR",
        targetId     : request.targetId,
        targetType   : "STUDENT",
        details      : request.reportDetails,
        status       : "PENDING"
    }

    result = insertReportIntoDatabase(newReport)

    IF result is NULL or failed:
        RETURN error "Failed to submit report"

    RETURN success "Report successfully submitted"

END FUNCTION
```

```
// INPUT:        request - contains request.sessionToken
// OUTPUT:       list of all reports or error message
// FUNCTIONALITY: Validates registrar authority then fetches and returns
//               all reports from the database including reporter, target, and status
FUNCTION viewAllReports(request):

    IF request is NULL:
        RETURN error "Invalid request"

    authLevel = returnAuthLevel(request.sessionToken)

    IF authLevel != REGISTRAR:
        RETURN error "Unauthorized - Registrar access required"

    reportList = fetchAllReports()

    IF reportList is NULL or empty:
        RETURN "No reports found"

    FOR EACH report IN reportList:
        reportCount = fetchReportCountByTarget(report.targetId)
        IF reportCount > 2:
            report.flagged = TRUE
        ELSE:
            report.flagged = FALSE

    RETURN reportList

END FUNCTION
```

```
// INPUT:        request - contains request.sessionToken, request.targetId
// OUTPUT:       success or error message
// FUNCTIONALITY: Validates registrar authority, checks that the student has more than
//               2 reports, then issues a formal warning to the student
FUNCTION warnStudent(request):

    IF request is NULL:
        RETURN error "Invalid request"

    authLevel = returnAuthLevel(request.sessionToken)

    IF authLevel != REGISTRAR:
        RETURN error "Unauthorized - Registrar access required"

    IF request.targetId is NULL:
        RETURN error "No student specified"

    student = fetchUserById(request.targetId)

    IF student is NULL:
        RETURN error "Student not found"

    reportCount = fetchReportCountByTarget(request.targetId)

    IF reportCount <= 2:
        RETURN error "Student does not meet warning threshold - must have more than 2 reports"

    warning = {
        studentId : request.targetId,
        reason    : "MULTIPLE_REPORTS",
        issuedBy  : request.userId,
        details   : "Student has received more than 2 reports"
    }

    result = insertWarningIntoDatabase(warning)

    IF result is NULL or failed:
        RETURN error "Failed to issue warning"

    SEND notifyStudent(request.targetId, "WARNING_ISSUED")

    RETURN success "Warning successfully issued to student"

END FUNCTION
```

```
// INPUT:        request - contains request.sessionToken, request.reportId, request.reporterId
// OUTPUT:       success or error message
// FUNCTIONALITY: Validates registrar authority, marks the specified report as false,
//               then issues a false reporting warning to the original reporter
FUNCTION warnFalseReporter(request):

    IF request is NULL:
        RETURN error "Invalid request"

    authLevel = returnAuthLevel(request.sessionToken)

    IF authLevel != REGISTRAR:
        RETURN error "Unauthorized - Registrar access required"

    IF request.reportId is NULL:
        RETURN error "No report specified"

    report = fetchReportById(request.reportId)

    IF report is NULL:
        RETURN error "Report not found"

    result = updateReportStatus(request.reportId, "FALSE_REPORT")

    IF result is NULL or failed:
        RETURN error "Failed to mark report as false"

    warning = {
        studentId : report.reporterId,
        reason    : "FALSE_REPORT",
        issuedBy  : request.userId,
        details   : "Student submitted a false report"
    }

    result = insertWarningIntoDatabase(warning)

    IF result is NULL or failed:
        RETURN error "Failed to issue false report warning"

    SEND notifyStudent(report.reporterId, "FALSE_REPORT_WARNING")

    RETURN success "Report marked as false and warning issued to reporter"

END FUNCTION
```

```
// INPUT:        authLevel - the role of the current user
//               userList - list of professors and students available to report
// OUTPUT:       rendered report submission form
// FUNCTIONALITY: Displays a form for students to submit a report against
//               a professor or another student, only visible to STUDENTS
FUNCTION displaySubmitReportAsStudent(authLevel, userList):

    IF authLevel != STUDENT:
        DISPLAY error "Unauthorized - Student access required"
        RETURN

    IF userList is NULL or empty:
        DISPLAY error "No users available to report"
        RETURN

    DISPLAY dropdown "Report Type" -> options ["PROFESSOR", "STUDENT"]

    ON reportTypeSelected:
        filteredUsers = FILTER userList WHERE user.role matches selectedReportType
        DISPLAY dropdown "Select Target" -> options FROM filteredUsers

    DISPLAY input field "Report Details"

    DISPLAY button "Submit Report" -> onClick SEND submitReportAsStudent(targetId, targetType, reportDetails)

END FUNCTION
```

```
// INPUT:        authLevel - the role of the current user
//               studentList - list of students available to report
// OUTPUT:       rendered report submission form
// FUNCTIONALITY: Displays a form for instructors to submit a report against
//               a student, only visible to INSTRUCTORS
FUNCTION displaySubmitReportAsInstructor(authLevel, studentList):

    IF authLevel != INSTRUCTOR:
        DISPLAY error "Unauthorized - Instructor access required"
        RETURN

    IF studentList is NULL or empty:
        DISPLAY error "No students available to report"
        RETURN

    DISPLAY dropdown "Select Student" -> options FROM studentList

    DISPLAY input field "Report Details"

    DISPLAY button "Submit Report" -> onClick SEND submitReportAsInstructor(targetId, reportDetails)

END FUNCTION
```

```
// INPUT:        authLevel - the role of the current user
//               reportList - list of all reports fetched from backend
// OUTPUT:       rendered report management panel
// FUNCTIONALITY: Displays all reports to the registrar, flags students with more than
//               2 reports and provides warn and false report buttons for each entry,
//               only visible to REGISTRAR
FUNCTION displayAllReports(authLevel, reportList):

    IF authLevel != REGISTRAR:
        DISPLAY error "Unauthorized - Registrar access required"
        RETURN

    IF reportList is NULL or empty:
        DISPLAY "No reports found"
        RETURN

    FOR EACH report IN reportList:
        DISPLAY report.reporterType + " - " + report.reporterId
        DISPLAY "Reported: " + report.targetType + " - " + report.targetId
        DISPLAY "Details: " + report.details
        DISPLAY "Status: " + report.status

        IF report.flagged == TRUE:
            DISPLAY "⚠ WARNING: This student has more than 2 reports"
            DISPLAY button "Warn Student" -> onClick SEND warnStudent(report.targetId)

        DISPLAY button "Mark as False Report" -> onClick SEND warnFalseReporter(report.reportId, report.reporterId)

    END FOR

END FUNCTION
```

```
// INPUT:        request - contains request.sessionToken
// OUTPUT:       full list of all admitted students or error message
// FUNCTIONALITY: Validates registrar authority then fetches and returns
//               the complete list of all admitted students from the main database
FUNCTION fetchAllStudents(request):

    IF request is NULL:
        RETURN error "Invalid request"

    authLevel = returnAuthLevel(request.sessionToken)

    IF authLevel != REGISTRAR:
        RETURN error "Unauthorized - Registrar access required"

    studentList = fetchAllStudentsFromDatabase()

    IF studentList is NULL or empty:
        RETURN error "No students found"

    RETURN studentList

END FUNCTION
```

```
// INPUT:        request - contains request.sessionToken, request.studentId,
//               request.notificationMessage
// OUTPUT:       success or error message
// FUNCTIONALITY: Validates registrar authority, verifies the student exists,
//               then sends a notification to the specified student to meet with the registrar
FUNCTION notifyStudentToMeet(request):

    IF request is NULL:
        RETURN error "Invalid request"

    authLevel = returnAuthLevel(request.sessionToken)

    IF authLevel != REGISTRAR:
        RETURN error "Unauthorized - Registrar access required"

    IF request.studentId is NULL:
        RETURN error "No student specified"

    IF request.notificationMessage is NULL or empty:
        RETURN error "Notification message is required"

    student = fetchUserById(request.studentId)

    IF student is NULL:
        RETURN error "Student not found"

    IF student.role != "STUDENT":
        RETURN error "Target is not a student"

    result = SEND notifyStudent(request.studentId, "MEETING_REQUESTED", request.notificationMessage)

    IF result is NULL or failed:
        RETURN error "Failed to send notification"

    RETURN success "Notification successfully sent to " + student.name

END FUNCTION
```

```
// INPUT:        request - contains request.sessionToken, request.studentId
// OUTPUT:       transcript data for the specified student or error message
// FUNCTIONALITY: Validates registrar authority, verifies the student exists,
//               then fetches and returns the full academic transcript for that student
FUNCTION fetchStudentTranscript(request):

    IF request is NULL:
        RETURN error "Invalid request"

    authLevel = returnAuthLevel(request.sessionToken)

    IF authLevel != REGISTRAR:
        RETURN error "Unauthorized - Registrar access required"

    IF request.studentId is NULL:
        RETURN error "No student specified"

    student = fetchUserById(request.studentId)

    IF student is NULL:
        RETURN error "Student not found"

    IF student.role != "STUDENT":
        RETURN error "Target is not a student"

    transcript = fetchTranscriptFromDatabase(request.studentId)

    IF transcript is NULL or empty:
        RETURN error "No transcript found for student"

    RETURN transcript

END FUNCTION
```

```
// INPUT:        authLevel - the role of the current user
//               studentList - full list of admitted students from backend
// OUTPUT:       rendered student records panel with notify and transcript buttons
// FUNCTIONALITY: Displays the full list of admitted students to the registrar,
//               each entry has a notify button and a view transcript button,
//               only visible to REGISTRAR
FUNCTION displayStudentRecords(authLevel, studentList):

    IF authLevel != REGISTRAR:
        DISPLAY error "Unauthorized - Registrar access required"
        RETURN

    IF studentList is NULL or empty:
        DISPLAY "No students found"
        RETURN

    FOR EACH student IN studentList:
        DISPLAY student.name
        DISPLAY student.studentId
        DISPLAY student.email
        DISPLAY student.GPA

        DISPLAY button "Notify to Meet" -> onClick DISPLAY notifyStudentModal(student.studentId)
        DISPLAY button "View Transcript" -> onClick SEND fetchStudentTranscript(student.studentId)

    END FOR

END FUNCTION
```

```
// INPUT:        authLevel - the role of the current user
//               studentId - the ID of the student to notify
// OUTPUT:       rendered modal with a message input and send button
// FUNCTIONALITY: Displays a modal popup allowing the registrar to compose
//               and send a meeting notification to a specific student,
//               only visible to REGISTRAR
FUNCTION displayNotifyStudentModal(authLevel, studentId):

    IF authLevel != REGISTRAR:
        DISPLAY error "Unauthorized - Registrar access required"
        RETURN

    IF studentId is NULL:
        DISPLAY error "No student specified"
        RETURN

    DISPLAY "Send Meeting Notification"
    DISPLAY input field "Message"

    DISPLAY button "Send Notification" -> onClick SEND notifyStudentToMeet(studentId, message)
    DISPLAY button "Cancel"            -> onClick CLOSE modal

END FUNCTION
```

```
// INPUT:        authLevel - the role of the current user
//               transcript - transcript data returned from backend
// OUTPUT:       rendered transcript view for the selected student
// FUNCTIONALITY: Displays the full academic transcript for a selected student
//               including all courses, grades, and GPA, only visible to REGISTRAR
FUNCTION displayStudentTranscript(authLevel, transcript):

    IF authLevel != REGISTRAR:
        DISPLAY error "Unauthorized - Registrar access required"
        RETURN

    IF transcript is NULL or empty:
        DISPLAY error "No transcript found for this student"
        RETURN

    DISPLAY "Transcript for: " + transcript.studentName
    DISPLAY "Student ID: "     + transcript.studentId
    DISPLAY "Cumulative GPA: " + transcript.GPA

    FOR EACH entry IN transcript.courseHistory:
        DISPLAY entry.courseCode
        DISPLAY entry.courseName
        DISPLAY entry.semester
        DISPLAY entry.grade
    END FOR

END FUNCTION
```

```
// INPUT:     studentId, courseId, letterGrade from form fields
// OUTPUT:    Displays success confirmation or error message
// FUNCTION:  Collects grade submission fields and sends to backend, handles response status codes
FUNCTION handleGradeSubmit:
    studentId   = getFieldValue("studentId")
    courseId    = getFieldValue("courseId")
    letterGrade = getFieldValue("letterGrade")

    IF ANY field is empty:
        displayError("All fields required")
        RETURN

    IF letterGrade NOT IN { A, B, C, D, F }:
        displayError("Invalid grade")
        RETURN

    response = SEND POST /api/grades/submit WITH { studentId, courseId, letterGrade }

    IF response.status == 200:
        displaySuccess("Grade submitted successfully")
    ELSE IF response.status == 403:
        displayError("Not authorized or grading period is closed")
    ELSE IF response.status == 404:
        displayError("Student or course not found")
    ELSE:
        displayError("Unexpected error")
END FUNCTION
```

```
// INPUT:     request { studentId, courseId, letterGrade }, instructorId from session token
// OUTPUT:    HTTP status code, on success: { gradeId, studentId, courseId, letterGrade }
// FUNCTION:  Verifies grading period is open, confirms instructor owns the course,
//            confirms student is enrolled, then inserts or updates grade record
FUNCTION submitGrade(request):
    instructorId = session.userId
    studentId    = request.studentId
    courseId     = request.courseId
    letterGrade  = request.letterGrade

    IF ANY field is null:
        RETURN 400

    IF letterGrade NOT IN { A, B, C, D, F }:
        RETURN 400

    // Check grading period
    IF DB.isGradingPeriodOpen() is FALSE:
        RETURN 403

    // Confirm instructor teaches this course
    course = DB.findCourseById(courseId)

    IF course is null:
        RETURN 404

    IF course.instructorId != instructorId:
        RETURN 403

    // Confirm student is enrolled in this course
    enrollment = DB.findEnrollment(studentId, courseId)

    IF enrollment is null:
        RETURN 404

    // Insert or update grade
    existingGrade = DB.findGrade(studentId, courseId)

    IF existingGrade is null:
        DB.saveGrade({ studentId, courseId, letterGrade })
    ELSE:
        DB.updateGrade(existingGrade.id, letterGrade)

    RETURN 200 WITH { studentId, courseId, letterGrade }
END FUNCTION
```

```
// INPUT:        request - contains request.sessionToken, request.query
// FUNCTIONALITY: Receives a user query, first searches the vector database
//               for a matching answer, if a confident match is found it returns
//               that answer, otherwise falls back to the LLM to generate a response
//               and attaches a hallucination warning to the result
FUNCTION handleChatbotQuery(request):

    IF request is NULL:
        RETURN error "Invalid request"

    authLevel = returnAuthLevel(request.sessionToken)

    IF authLevel == VISITOR:
        RETURN error "Unauthorized - Please log in"

    IF request.query is NULL or empty:
        RETURN error "No query provided"

    vectorResult = searchVectorDatabase(request.query)

    IF vectorResult is NOT NULL AND vectorResult.confidenceScore >= CONFIDENCE_THRESHOLD:
        response = {
            answer              : vectorResult.answer,
            source              : "VECTOR_DATABASE",
            hallucinationWarning: FALSE
        }
        RETURN response

    ELSE:
        llmResult = callLLM(request.query)

        IF llmResult is NULL or failed:
            RETURN error "Failed to generate a response"

        response = {
            answer              : llmResult.answer,
            source              : "LLM_GENERATED",
            hallucinationWarning: TRUE,
            warningMessage      : "This response was generated by an AI language model and
                                   could not be verified against our knowledge base.
                                   Please verify this information with an official source."
        }
        RETURN response

END FUNCTION
```

```
// INPUT:        query - the user's question as a string
// OUTPUT:       vectorResult - contains vectorResult.answer and vectorResult.confidenceScore
// FUNCTIONALITY: Converts the query into an embedding, searches the vector database
//               for the closest matching entry, returns the result and its confidence score
FUNCTION searchVectorDatabase(query):

    IF query is NULL or empty:
        RETURN NULL

    queryEmbedding = generateEmbedding(query)

    IF queryEmbedding is NULL or failed:
        RETURN NULL

    vectorResult = queryVectorDatabase(queryEmbedding)

    IF vectorResult is NULL:
        RETURN NULL

    RETURN {
        answer         : vectorResult.answer,
        confidenceScore: vectorResult.similarityScore
    }

END FUNCTION
```

```
// INPUT:     request { action }, registrarId from session token
// OUTPUT:    HTTP status code, on success: { gradingPeriodStatus }
// FUNCTION:  Allows registrar to manually open or close the grading period
FUNCTION setGradingPeriod(request):
    registrarId = session.userId
    action      = request.action  // "OPEN" or "CLOSE"

    IF session.role != REGISTRAR:
        RETURN 403

    IF action NOT IN { OPEN, CLOSE }:
        RETURN 400

    IF action == OPEN:
        DB.setGradingPeriodOpen(TRUE)
    ELSE:
        DB.setGradingPeriodOpen(FALSE)

    RETURN 200 WITH { gradingPeriodStatus: action }
END FUNCTION
```

```
// INPUT:        authLevel - the role of the current user
// OUTPUT:       rendered chatbot panel with message history and input field
// FUNCTIONALITY: Displays the chatbot interface, sends user queries to the backend,
//               renders responses and displays a hallucination warning banner
//               if the response was LLM generated
FUNCTION displayChatbot(authLevel):

    IF authLevel == VISITOR:
        DISPLAY error "Unauthorized - Please log in"
        RETURN

    DISPLAY chatWindow
    DISPLAY messageHistory

    DISPLAY input field "Ask a question..."
    DISPLAY button "Send" -> onClick SEND handleChatbotQuery(inputField.value)

    ON responseReceived:
        DISPLAY response.answer

        IF response.source == "VECTOR_DATABASE":
            DISPLAY "✓ Answer sourced from verified knowledge base"

        ELSE IF response.source == "LLM_GENERATED":
            DISPLAY "⚠ Hallucination Warning: " + response.warningMessage

END FUNCTION
```

```
// INPUT:     None (studentId pulled from session token)
// OUTPUT:    Displays graduation confirmation, warning, or error message
// FUNCTION:  Sends graduation application request to backend, handles response status codes
FUNCTION handleGraduationApplication:

    response = SEND POST /api/graduation/apply

    IF response.status == 200:
        displaySuccess("Congratulations! You have graduated. Farewell!")
        redirectToLogout()
    ELSE IF response.status == 403:
        displayWarning("You do not meet graduation requirements. " +
                       "Completed: " + response.completedClasses + "/8 classes.")
    ELSE IF response.status == 409:
        displayError("Graduation application already pending.")
    ELSE:
        displayError("Unexpected error")
END FUNCTION
```

```
// INPUT:     studentId from session token
// OUTPUT:    HTTP status code, on success: { message }, on failure: { completedClasses, requiredClasses }
// FUNCTION:  Verifies student role, checks graduation requirements, grants degree and
//            deletes student record if met, warns student if not met
FUNCTION applyForGraduation(request):
    studentId = session.userId

    IF session.role != STUDENT:
        RETURN 403

    // Fetch student record
    student = DB.findUserById(studentId)

    IF student is null:
        RETURN 404

    // Check completed classes
    completedClasses = DB.countCompletedCourses(studentId)

    IF completedClasses < 8:
        RETURN 403 WITH {
            "message":          "Requirements not met",
            "completedClasses": completedClasses,
            "requiredClasses":  8
        }

    // Requirements met — award degree and delete record
    DB.saveGraduationRecord({
        firstName:   student.firstName,
        lastName:    student.lastName,
        targetMajor: student.targetMajor,
        GPA:         student.GPA,
        graduatedAt: currentTimestamp()
    })

    DB.deleteUser(studentId)

    RETURN 200 WITH { "message": "Graduation approved. Record archived." }
END FUNCTION
```

```
// INPUT:     targetCollegeId from form field
// OUTPUT:    Displays success confirmation or error message
// FUNCTION:  Collects target college ID and sends transfer application to backend,
//            handles response status codes
FUNCTION handleTransferApplication:
    targetCollegeId = getFieldValue("targetCollegeId")

    IF targetCollegeId is empty:
        displayError("Target college ID is required")
        RETURN

    response = SEND POST /api/transfer/apply WITH { targetCollegeId }

    IF response.status == 201:
        displaySuccess("Transfer application submitted. Awaiting registrar approval.")
    ELSE IF response.status == 404:
        displayError("Target college not found.")
    ELSE IF response.status == 409:
        displayError("Transfer application already pending.")
    ELSE:
        displayError("Unexpected error")
END FUNCTION
```

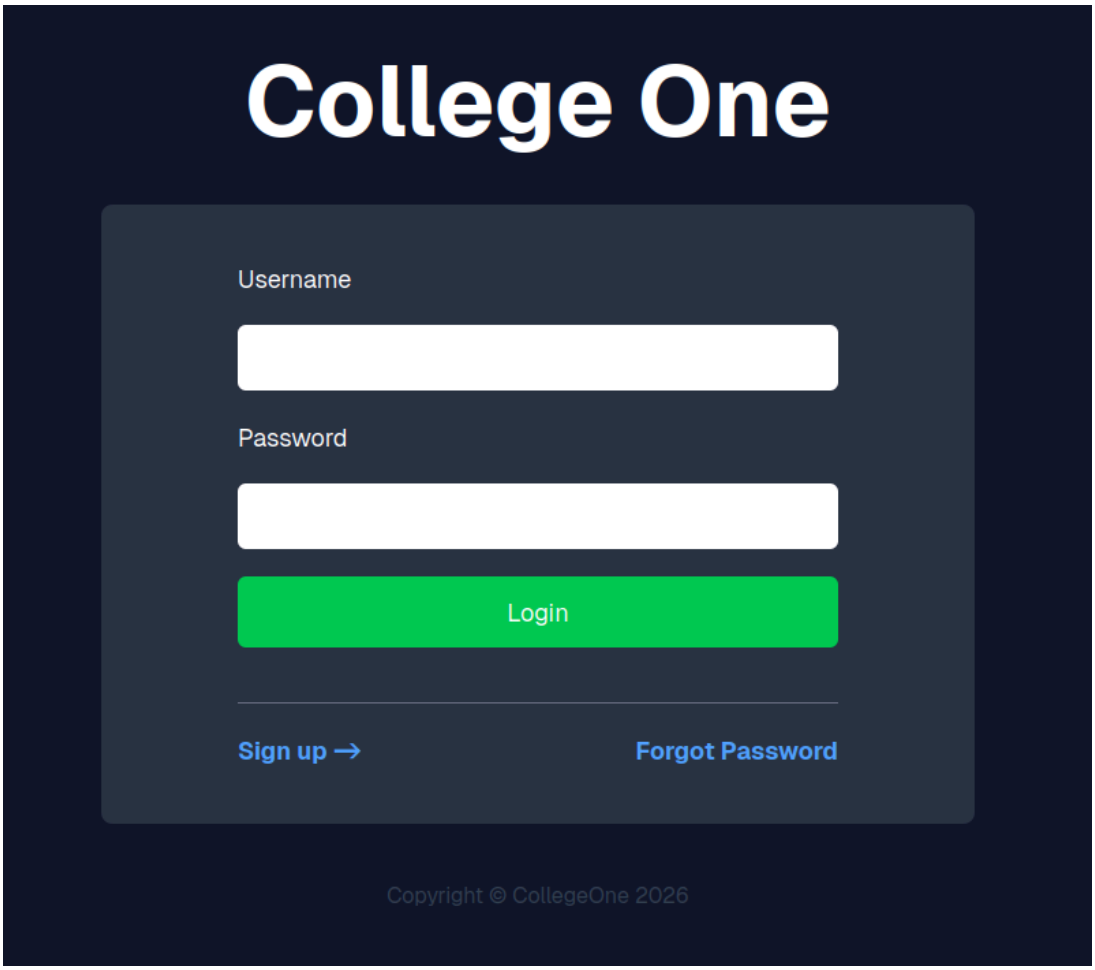
```
// INPUT:     applicationId, action from form fields
// OUTPUT:    Displays success confirmation or error message
// FUNCTION:  Allows registrar to approve or deny a pending transfer application
FUNCTION handleTransferDecision:
    applicationId = getFieldValue("applicationId")
    action        = getFieldValue("action")   // "APPROVE" or "DENY"

    IF ANY field is empty:
        displayError("All fields required")
        RETURN

    response = SEND POST /api/transfer/decide WITH { applicationId, action }

    IF response.status == 200:
        displaySuccess("Transfer application " + action + "D successfully.")
    ELSE IF response.status == 403:
        displayError("Not authorized.")
    ELSE IF response.status == 404:
        displayError("Application not found.")
    ELSE:
        displayError("Unexpected error")
END FUNCTION
```

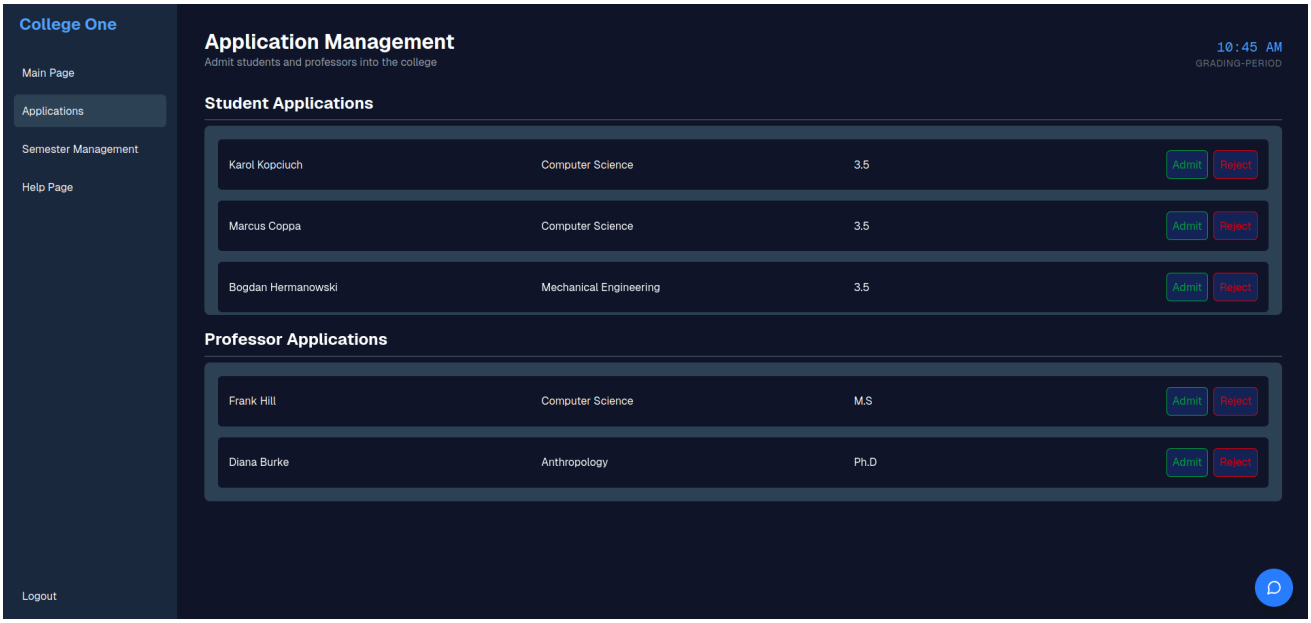
```
// INPUT:     request { targetCollegeId }, studentId from session token
// OUTPUT:    HTTP status code, on success: { applicationId, status: PENDING }
// FUNCTION:  Validates student role, checks target college exists, checks no
//            pending application exists, creates transfer application record
FUNCTION applyForTransfer(request):
    studentId       = session.userId
    targetCollegeId = request.targetCollegeId

    IF session.role != STUDENT:
        RETURN 403

    IF targetCollegeId is null:
        RETURN 400

    // Check target college exists
    targetCollege = DB.findCollegeById(targetCollegeId)

    IF targetCollege is null:
        RETURN 404

    // Check no pending application already exists
    existingApplication = DB.findPendingTransfer(studentId)

    IF existingApplication is NOT null:
        RETURN 409

    // Create transfer application
    application = {
        studentId:       studentId,
        targetCollegeId: targetCollegeId,
        status:          PENDING,
        submittedAt:     currentTimestamp()
    }

    DB.saveTransferApplication(application)

    RETURN 201 WITH { applicationId: application.id, status: PENDING }
END FUNCTION
```

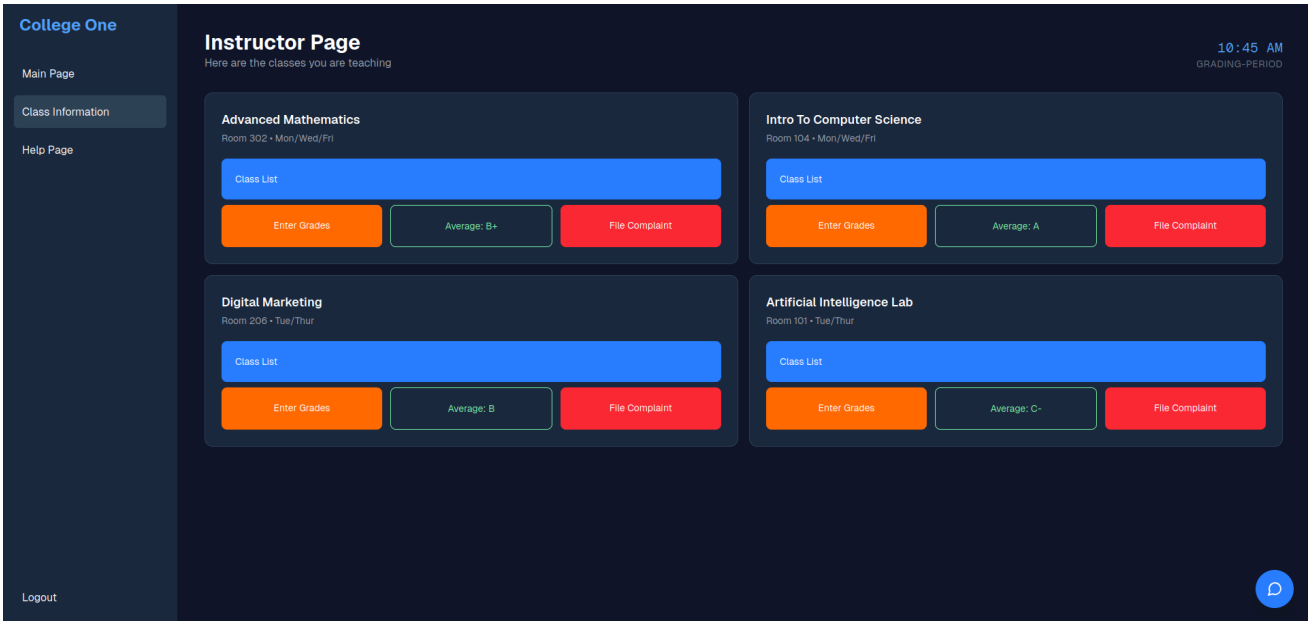
```
// INPUT:     request { applicationId, action }, registrarId from session token
// OUTPUT:    HTTP status code, on success: { applicationId, status }
// FUNCTION:  Validates registrar role, fetches application, on approval copies student
//            record to target college and deletes from current, on denial updates status only
FUNCTION decideTransfer(request):
    registrarId   = session.userId
    applicationId = request.applicationId
    action        = request.action   // "APPROVE" or "DENY"

    IF session.role != REGISTRAR:
        RETURN 403

    IF action NOT IN { APPROVE, DENY }:
        RETURN 400

    // Fetch application
    application = DB.findTransferApplicationById(applicationId)

    IF application is null:
        RETURN 404

    IF application.status != PENDING:
        RETURN 409

    IF action == DENY:
        DB.updateTransferStatus(applicationId, DENIED)
        RETURN 200 WITH { applicationId, status: DENIED }

    // action == APPROVE
    student = DB.findUserById(application.studentId)

    IF student is null:
        RETURN 404

    // Copy record to target college
    DB.saveStudentToCollege(application.targetCollegeId, student_info)

    // Delete from current college
    DB.deleteUser(application.studentId)
    DB.updateTransferStatus(applicationId, APPROVED)

    RETURN 200 WITH { applicationId, status: APPROVED }
END FUNCTION
```

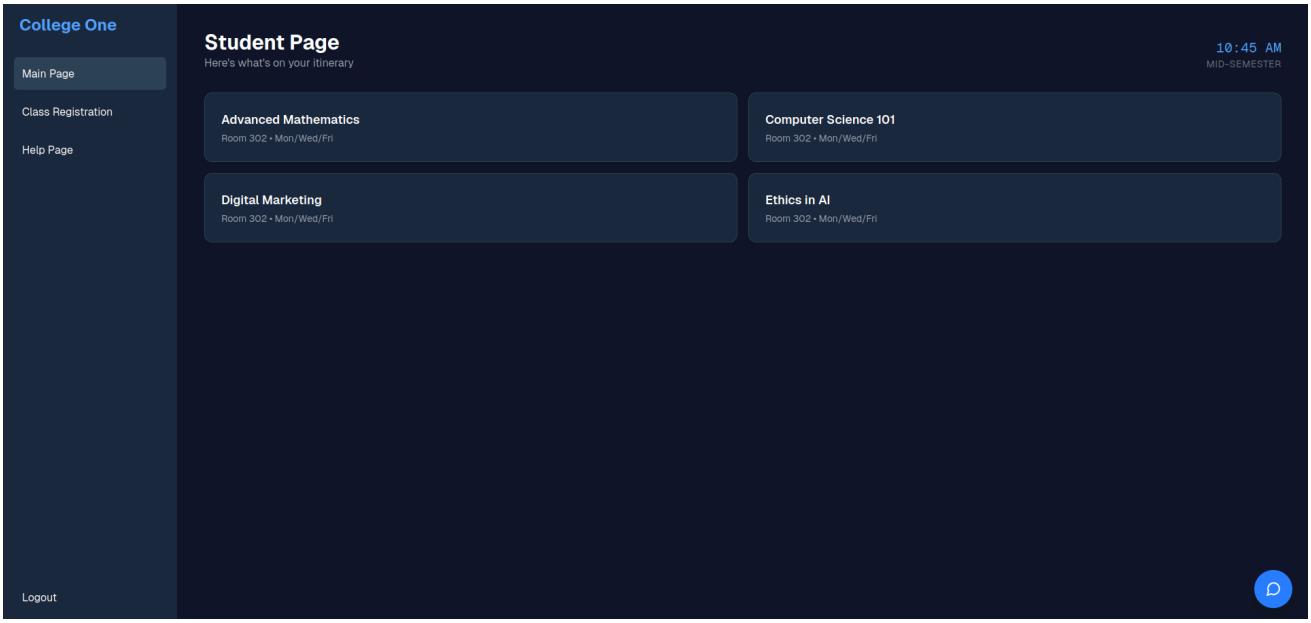
5. System Screens

5.1 Major GUI Screens

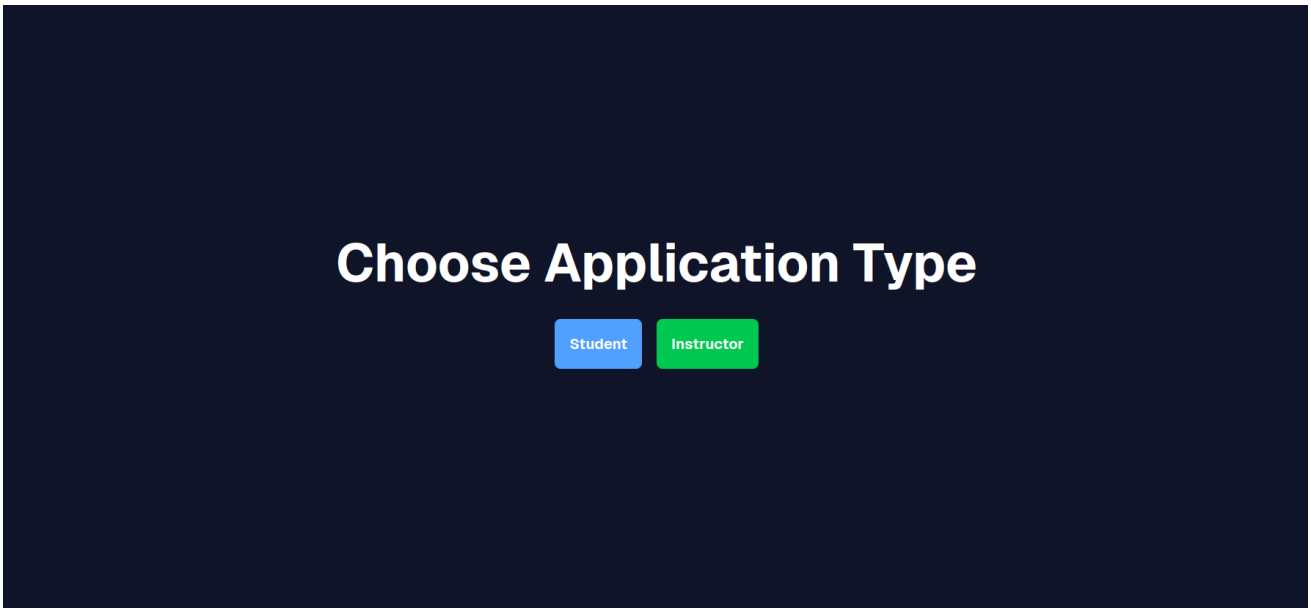


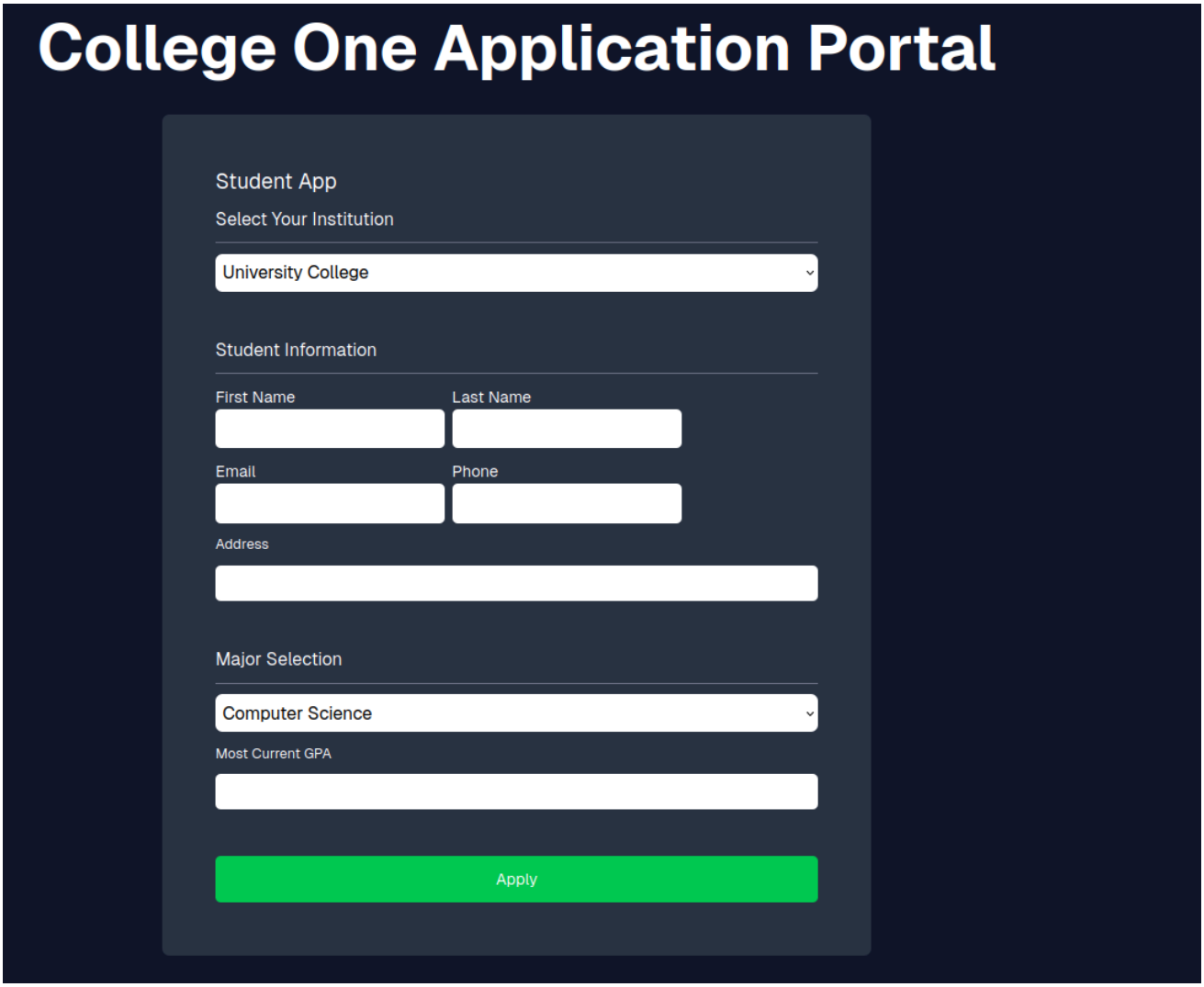


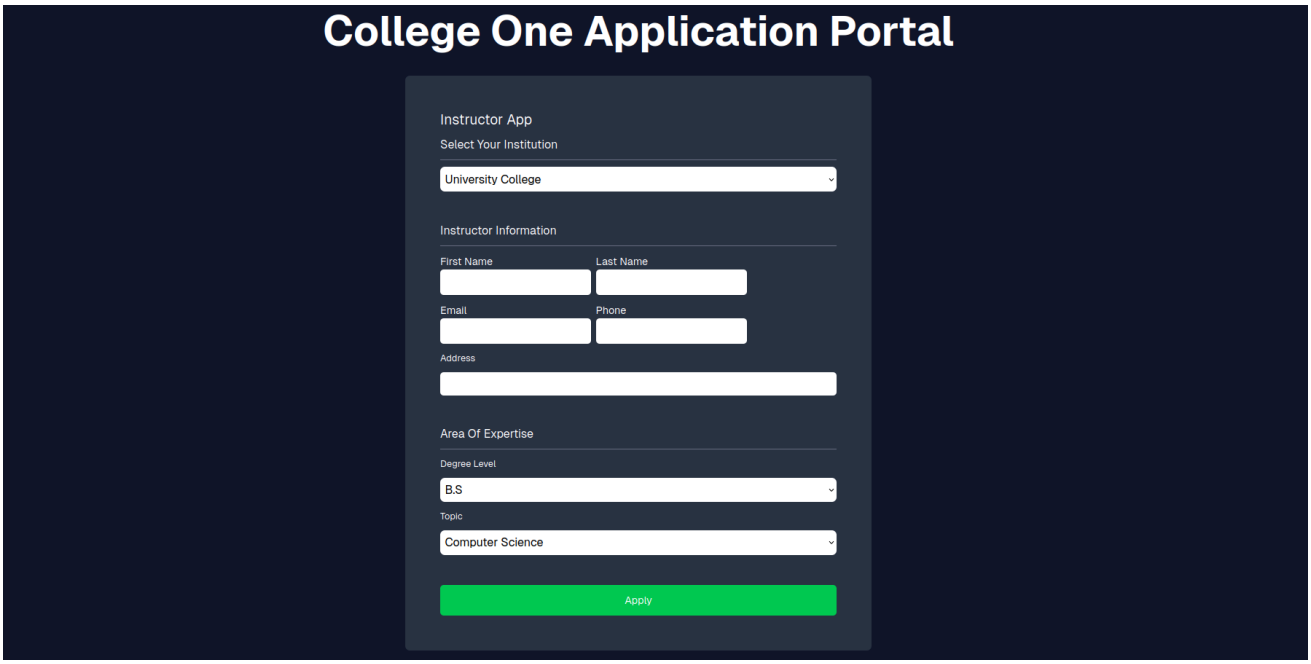

College One
Main Page
Class Information
Help Page
Logout
Instructor Page
Here are the classes you are teaching
10:45 AM
GRADING-PERIOD
Advanced Mathematics
Room 302 • Mon/Wed/Fri
Class List
Enter Grades
Average: B+
File Complaint
Intro To Computer Science
Room 104 • Mon/Wed/Fri
Class List
Enter Grades
Average: A
File Complaint
Digital Marketing
Room 206 • Tue/Thur
Class List
Enter Grades
Average: B
File Complaint
Artificial Intelligence Lab
Room 101 • Tue/Thur
Class List
Enter Grades
Average: C-
File Complaint


College One
Main Page
Class Registration
Help Page
Logout
Student Page
Here's what's on your itinerary
10:45 AM
MID-SEMESTER
Advanced Mathematics
Room 302 • Mon/Wed/Fri
Computer Science 101
Room 302 • Mon/Wed/Fri
Digital Marketing
Room 302 • Mon/Wed/Fri
Ethics in AI
Room 302 • Mon/Wed/Fri

5.2 Sample Prototype







Following these pages, any visitor can sign up and log into the college one webpage

6. Group Meeting Memos

Website Architecture Meeting

- Met to discuss various aspects of the websites architecture and what technologies will be implemented
- Decided on producing a NextJS based frontend, as well as a Spring Boot Application to hold data on the backend
- Produced general layout for how the website will be structured for the final product
- Discussed what design choices we would need to make to make sure that the website looks and functions

Memo For Upcoming time

- Begin establishing connection between the backend API as well as the frontend javascript code
- Start to test certain initial aspects of the system overall in order to catch any critical bugs prior to the rollout of the first test version
- Begin simple data seeding in order to make sure that our database will have the proper entities stored over time

7. GitHub Repository

https://github.com/LOLFUTUREVEVO/College0